Ministry of Education and Science of Ukraine

National University of "Kyiv-Mohyla Academy"

Department of Computer Science of the Faculty of Informatics

Optimization of Genetic Algorithm Computations through CUDA Parallel Processing

Text part of the course paper in the specialty

"Computer Science" – 122

Thesis Supervisor:

Senior Lecturer

Sergii Medvid

_____ (Signature)

" 24 " March 2026

Done by Student:

Anastasiia Aleksieienko

_____ (Signature)

" 24 " March 2026

Kyiv, 2026

Contents

Анотація **3**

Abstract **4**

1 Introduction **5**

2 Theoretical Background **7**

2.1 GPU-GA Literature Review 7

2.1.1 Genetic Algorithms Parallelization Potential 7

2.1.2 CUDA Model Suitability for GA-Specific Computations . . . 9

2.1.3 Prior GPU–GA Studies on GA Core Operations Optimization 11

2.1.4 Implementation Challenges and Scalability Limits 12

2.2 CUDA Architecture and Kernel Execution Model 15

2.2.1 CUDA Architecture Overview and Fitness Kernel Design . . . 15

2.2.2 GPU Occupancy and Utilization 18

2.2.3 Expected Speedup Bounds from Amdahl's Law Application . 20

3 Experimental Methodology and Results **22**

3.1 Experimental Methodology 22

3.1.1 Experiment Conduction and Benchmarks 22

3.1.2 Optimal Device Occupancy Calculation 29

3.2 XOR Perceptron Benchmark 32

3.2.1 Performance Analysis 32

3.2.2 Results 33

3.2.3 Scaling Analysis 33

3.2.4 Statistical Significance 35

3.3 TSP Benchmark 37

3.3.1 Performance Analysis 37

3.3.2 Results 38

3.3.3 Scaling Analysis 38
3.3.4 Statistical Significance and Power Analysis 40
3.4 Results Interpretation and Implications 41

4 Framework Development **43**
4.1 Library Design and Architecture 43
4.2 Experiments and Benchmarks Results Comparison 46

5 Conclusion **54**

A Schedule **56**

Б Assignment **57**

C Library Source Code **59**

Анотація

Алексєєнко А. Д. Оптимізація обчислень генетичних алгоритмів через паралельну обробку за допомогою CUDA. - Курсова робота. Національний університет «Києво-Могилянська академія», факультет інформатики, кафедра інформатики. - Київ, 2026.

У роботі досліджується прискорення генетичних алгоритмів (ГА) через паралелізацію на GPU з використанням CUDA та Python-нативних ядер, JIT-компільованих бібліотекою Numba. Розроблено CUDA-ядра для основних операторів ГА - обчислення пристосованості, селекції, схрещування та мутації - та проведено порівняння з CPU-реалізаціями на задачах навчання XOR-перцептрона і комівояжера (TSP).

GPU-реалізації перевершують CPU при популяції понад 100 особин, досягаючи прискорення від 1,15× до 14,56× на NVIDIA RTX 2050. Усі відмінності статистично значущі ($p < 0{,}05$, Cohen's $d > 0{,}8$). Домінуючим вузьким місцем є передача даних між хостом і пристроєм, що вказує на можливість подальшого прискорення за рахунок GPU-резидентних еволюційних циклів. Кодову базу реорганізовано у публічно доступну Python-бібліотеку для розробки ГА із GPU-прискоренням.

Ключові слова: генетичні алгоритми, CUDA, паралелізація на GPU, Numba, оптимізація продуктивності, задача комівояжера, навчання нейронних мереж, Python, паралельні обчислення.

Abstract

Aleksieienko A. D. Optimization of Genetic Algorithm Computations through CUDA Parallel Processing. - Course paper. National University of "Kyiv-Mohyla Academy", Faculty of Informatics, Department of Computer Science. - Kyiv, 2026.

This paper investigates the acceleration of genetic algorithms (GAs) through CUDA-based GPU parallelization using Python-native Numba JIT-compiled kernels. Custom kernels for fitness evaluation, selection, crossover, and mutation were benchmarked against CPU implementations on XOR perceptron training and the Traveling Salesman Problem.

GPU implementations outperform CPU baselines for populations exceeding 100 individuals, achieving speedups of 1.15× to 14.56× on an NVIDIA RTX 2050. All differences are statistically significant ($p < 0.05$, Cohen's $d > 0.8$). The dominant bottleneck is host-device data transfer, indicating further gains are achievable with fully GPU-resident evolutionary loops. The codebase was refactored into a publicly available Python library for GPU-accelerated GA development.

Keywords: genetic algorithms, CUDA, GPU parallelization, Numba, performance optimization, traveling salesman problem, neural network training, Python, parallel computing.

1 Introduction

This course paper investigates the acceleration of genetic algorithms through CUDA-based GPU parallelization, covering both performance analysis and the development of reusable implementation components.

Hypothesis. The application of CUDA technology to parallelize computations in genetic algorithms will enable significant acceleration of key operations: fitness function evaluation, selection, crossover, and mutation - compared to sequential implementation on the CPU. It is expected that parallelization efficiency will increase with population size and computational complexity of the fitness function, subject to the theoretical limits imposed by Amdahl's Law and data transfer overhead.

Parallel processing of large populations on the GPU is expected to improve the quality of solutions found, because it allows operation with significantly larger population sizes without a proportional increase in overall algorithm execution time. This opens up new possibilities for solving complex optimization problems in acceptable timeframes.

Object of the Research. Genetic algorithms represent the general domain of the research paper, serving as black-box optimization material.

Subject of the Research. Parallelization strategies and speedups achieved through CUDA form the specific focus, targeting custom kernel development for selection, crossover, mutation, and fitness evaluation on GPU.

Research Gap. A comparative analysis of publication volumes on Semantic Scholar[1] (accessed October 2025) shows that GPU-accelerated genetic algorithm

[1]Semantic Scholar Academic Graph API v1. Endpoint: `GET .../graph/v1/paper/search?query={Q}&limit=1` (base URL: `api.semanticscholar.org`). The `total` field in the JSON response gives the match count; `&year=2020-2025` was appended for five-year filtering. All queries: October 2025. Verification in March 2026: control returned `total: 286918` (all-time) and `total: 109069` (5-year), confirming proportionality with the original data.

optimization is substantially less studied than established optimization domains. Table 1 summarizes the result counts.

Table 1: Semantic Scholar publication counts by query (October 2025).

Query	All-time	5-year	Ratio*
databases clustering optimization (control)	243,000	94,500	1×
CUDA parallel processing genetic algorithms	8,800	1,380	28×
MapReduce parallel processing genetic algorithms optimization	7,530	1,150	32×
OpenCL parallel processing genetic algorithms	6,250	946	39×

*Ratio = control all-time / query all-time, rounded.

Within this space, the majority of GPU implementations use native C/C++ CUDA kernels or high-level framework abstractions (EvoJAX, EvoTorch, EvoX). Python-native CUDA kernel development via Numba, despite NVIDIA's investment in a Python CUDA SDK, appears rarely in the evolutionary computation literature. This gap motivates an investigation into whether Python-native GPU kernel development can achieve competitive speedups while retaining the prototyping speed and accessibility advantages that Python offers over lower-level CUDA C/C++ workflows.

Structure of the Paper. Section 2 provides the theoretical background: a literature review of GPU-accelerated genetic algorithms (Section 2.1) and an overview of CUDA architecture with kernel design principles (Section 2.2). Section 3 presents the experimental methodology, benchmark configurations, and results for two test problems: XOR perceptron training (Section 3.2) and the Traveling Salesman Problem (Section 3.3), followed by a cross-benchmark analysis (Section 3.4). Section 4 describes the reusable GPU-accelerated GA library developed from the experimental code and validates its results against the original benchmarks. Section 5 summarizes the findings and outlines directions for future work.

2 Theoretical Background

2.1 GPU-GA Literature Review

2.1.1 Genetic Algorithms Parallelization Potential

Genetic algorithms (GAs) are metaheuristic optimization techniques inspired by natural selection, which simulate evolutionary processes through key operations: fitness evaluation, selection, crossover, and mutation. GAs are stochastic global search methods that combine two strategies: exploiting better solutions and exploring the search space [6]. A GA maintains a population of candidate solutions (individuals), each encoded as a chromosome, and iteratively evolves them over generations. GAs belong to the class of black-box optimization methods where only objective function evaluations are required - no gradient information or problem-specific knowledge is needed [21]. The concept originates from John Henry Holland's work at the University of Michigan [16], whose "original goal was not to design algorithms to solve specific problems, but rather to formally study the phenomenon of adaptation as it occurs in nature" [28].

Applications and Problem Domains. GAs are effective for solving NP-hard problems such as the Traveling Salesman Problem (TSP), Vehicle Routing Problem (VRP), job-shop scheduling, and multi-objective optimization [12, 21]. They avoid local optima traps that complicate gradient-based methods and are suited for vast, discontinuous search spaces.

Core Operations of Genetic Algorithms. The GA lifecycle comprises four operations: **Fitness Evaluation** computes an objective function value for each individual independently - typically the main computational bottleneck. **Selection** chooses parents probabilistically based on fitness (e.g., tournament, roulette wheel). **Crossover** combines parent chromosomes to produce offspring (e.g., single-point, uniform). **Mutation** introduces random changes to maintain diversity (e.g., bit-flip, swap).

Each generation, n offspring are produced through selection, crossover (P_c), and mutation (P_m), then replace the current population [21]. Figure 1 illustrates this generational loop.

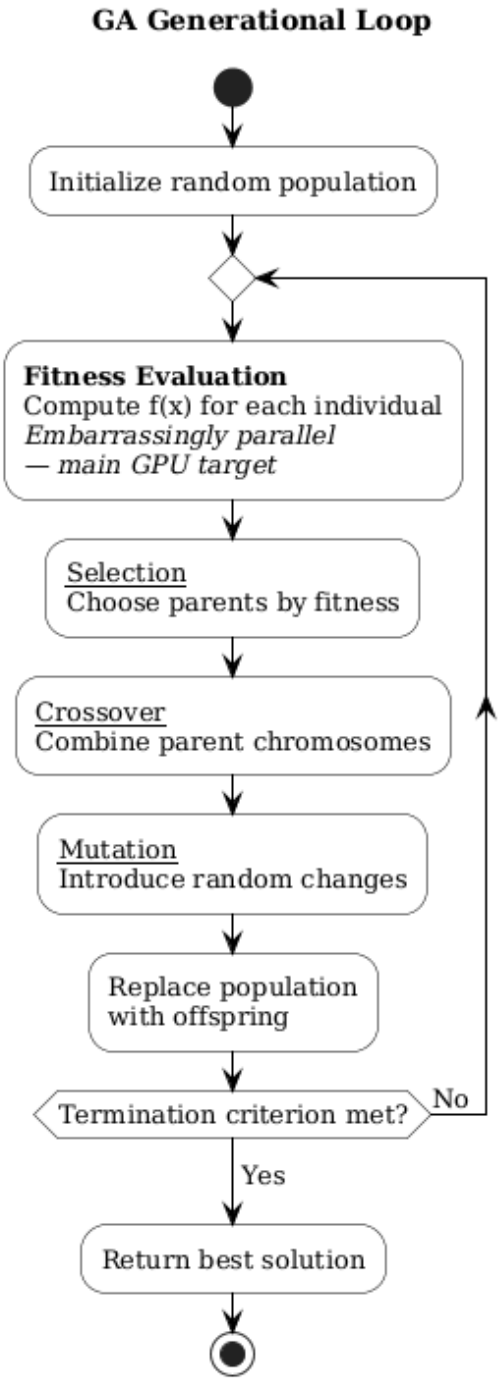


Figure 1: GA generational loop. Fitness evaluation is the primary GPU acceleration target due to its embarrassingly parallel nature.

Black-Box Optimization Nature and Parallelization Potential

GAs are black-box optimizers: they evaluate solutions solely through fitness function calls, requiring no gradient computation or analytical models [21]. "The GA does not require derivative information or any other auxiliary knowledge about the function being optimized" [28]. In formal terms, the objective function is treated as an opaque oracle returning $f(\mathbf{x})$ given input $\mathbf{x}$, without revealing gradients or analytical form [4, 29].

This black-box property directly enables parallelization: since fitness evaluation has no inter-individual dependencies, each individual can be evaluated in complete isolation. This transforms population-wide fitness evaluation into an embarrassingly parallel workload, the ideal case for GPU acceleration where thousands of threads execute identical computations on different individuals [6]. As Cheng and Gen note, "on GPU architecture, you can run it with more than hundreds of thousands of massive threads simultaneously" [6].

Fitness evaluation dominates sequential GA execution time: "the bottleneck of the GAs is the fitness evaluation task" [42], and can consume the majority of total computation time for complex fitness functions such as neural network training [42].

2.1.2 CUDA Model Suitability for GA-Specific Computations

CUDA Model Suitability for GA-Specific Computations. Compute Unified Device Architecture - "CUDA is a parallel computing platform and programming model developed by NVIDIA that enables dramatic increases in computing performance by harnessing the power of the GPU" [33]. CUDA organizes computation into kernels executed by lightweight threads arranged in blocks and grids, with access to a hierarchical memory system (global, shared, constant, and texture memories). This architecture is well suited to data-parallel problems where the same operation is applied independently across large sets of data - a

pattern that matches the population-based structure of GAs, where fitness evaluation and variation operators are applied repeatedly across individuals. The CUDA platform exposes GPU architectural features directly to programmers, enabling fine-grained control over thread execution to fully exploit GPU computing power [6].

Python-CUDA Integration for GA Development. The primary CUDA API uses C, but Python interfaces exist: CuPy provides NumPy-compatible GPU arrays backed by cuBLAS and cuFFT [40]; Numba provides a JIT compiler that generates CUDA kernels directly from Python via the `@cuda.jit` decorator [30]. These tools allow GA practitioners to implement operations in Python and delegate numeric workload to CUDA kernels on the GPU. "Numba exposes the CUDA programming model, just like in CUDA C/C++, but using pure python syntax, so that programmers can create custom, tuned parallel kernels without leaving the comforts and advantages of Python behind."[15]

Benefits and Drawbacks of Python for CUDA GA Development. Python is chosen for CUDA development in this paper to balance performance with rapid prototyping and accessibility. CuPy compiles custom kernels at runtime using NVRTC (NVIDIA Runtime Compilation), producing PTX that the driver compiles to device code, with results cached for reuse [40]. Numba's JIT achieves performance close to native CUDA: "CUDA Python Mandelbrot code runs nearly 1700 times faster than the pure Python version" [15]. Python's concise syntax and ecosystem make it effective for disseminating GPU-accelerated GA techniques beyond systems programming specialists.

First, Python's interpreter adds host-side overhead to every GPU call; for small inputs the launch overhead can exceed the computation itself [47]. Second, the Python-CUDA stack depends on multiple native libraries whose versions must align: CuPy requires a matched CUDA toolkit wheel [10], and Numba requires both the toolkit and a compatible driver [30]. Third, Python's GIL limits multi-core CPU orchestration of multiple GPUs, as it "prevents multiple threads from

executing Python code at the same time" [13]. These drawbacks are mitigated by focusing on large, compute-bound GA workloads with most computation on the device.

2.1.3 Prior GPU–GA Studies on GA Core Operations Optimization

This subsection reviews relevant studies to identify patterns in GPU-accelerated GA computing. The review also examines the Python libraries and tools used.

Study on Parallel Hybrid Genetic Algorithm and Machine Learning on GPU and Applications to Railway Scheduling. Cheng and Gen [6] parallelized a hybrid multi-objective GA applied to railway scheduling problems (ARL, BTS, and Japanese high-speed rail). The implementation offloaded differential evolution mutation, k-means clustering, and fitness evaluation to GPUs, which was necessary for maintaining practical computation times. The parallel implementation preserved solution quality while improving efficiency.

GPU-Accelerated NSGA-II for Crystallization via Numba CUDA. Rangavajhala et al. apply GPU-parallelized NSGA-II to crystallization process optimization, with the entire pipeline implemented using Numba CUDA kernels, achieving a 122× speedup over the CPU baseline [41].

GPU-Accelerated Stack-Based Genetic Programming: Sathia et al. Candidate expressions are represented as prefix lists evaluated using fixed-length stacks in GPU memory. Benchmarked on an NVIDIA DGX-A100, the implementation achieves 119× and 40× speedups over gplearn on synthetic and large-scale datasets respectively [45]. The algorithm has been integrated into cuML, NVIDIA's scikit-learn-compatible GPU library [45].

EvoGP: GPU-Accelerated Tree-Based Genetic Programming. Wu et al. present EvoGP, a GPU-accelerated framework for tree-based genetic programming that achieves up to 304× speedup over existing GPU-based TGP implementations and 18× over state-of-the-art CPU-based libraries [53].

GPU-Accelerated NSGA-III for Protein Encoding. Kim and Kim re-

port a 15,454× speedup over Pymoo, enabling scaling from 100 solutions over 100 cycles to 400 solutions over 12,800 cycles [22]. The speedup magnitude partly reflects the choice of baseline (Pymoo is a Python framework). NSGA-III surpassed NSGA-II beyond 3,200 cycles, showing that GPU acceleration enables both faster computation and better solution quality through more extensive search [22].

Conclusions and implications. The literature demonstrates that large speedups are achievable when GA operations are mapped to CUDA's thread hierarchy. Reported accelerations range from 18×–122× over CPU baselines [53, 41, 45] to 304× over prior GPU implementations [53] and 15,454× over Python frameworks [22], though speedup magnitudes depend on the choice of baseline. Fitness evaluation is the primary acceleration target, though parallel selection, crossover, and mutation can further reduce runtime. The dominant strategy involves CUDA C/C++ kernels with Python orchestration, whether through low-level bindings like PyCUDA or GPU-native libraries like cuML [45]. These findings inform this thesis's design, where fitness evaluation dominates GPU computation and genetic operators run as parallel kernels.

2.1.4 Implementation Challenges and Scalability Limits

Several factors can limit GPU performance advantages over CPUs. Small inputs leave most of the device idle, since thousands of concurrent threads are needed for efficient operation [47]. Low arithmetic intensity causes the GPU to spend most time waiting on memory, and the fixed cost of a kernel launch can exceed the computation [47]. Host-device data transfer adds PCIe latency, so data should be transferred once and kept resident [47]. Data types also matter: 64-bit floating-point throughput varies from 1/2 (data center GPUs like A100) to 1/64 (consumer GPUs like the RTX 2050 used in this work) of 32-bit throughput, depending on the architecture [35].

Figure 2 shows how these bottlenecks arise in a typical GPU-accelerated GA execution loop.

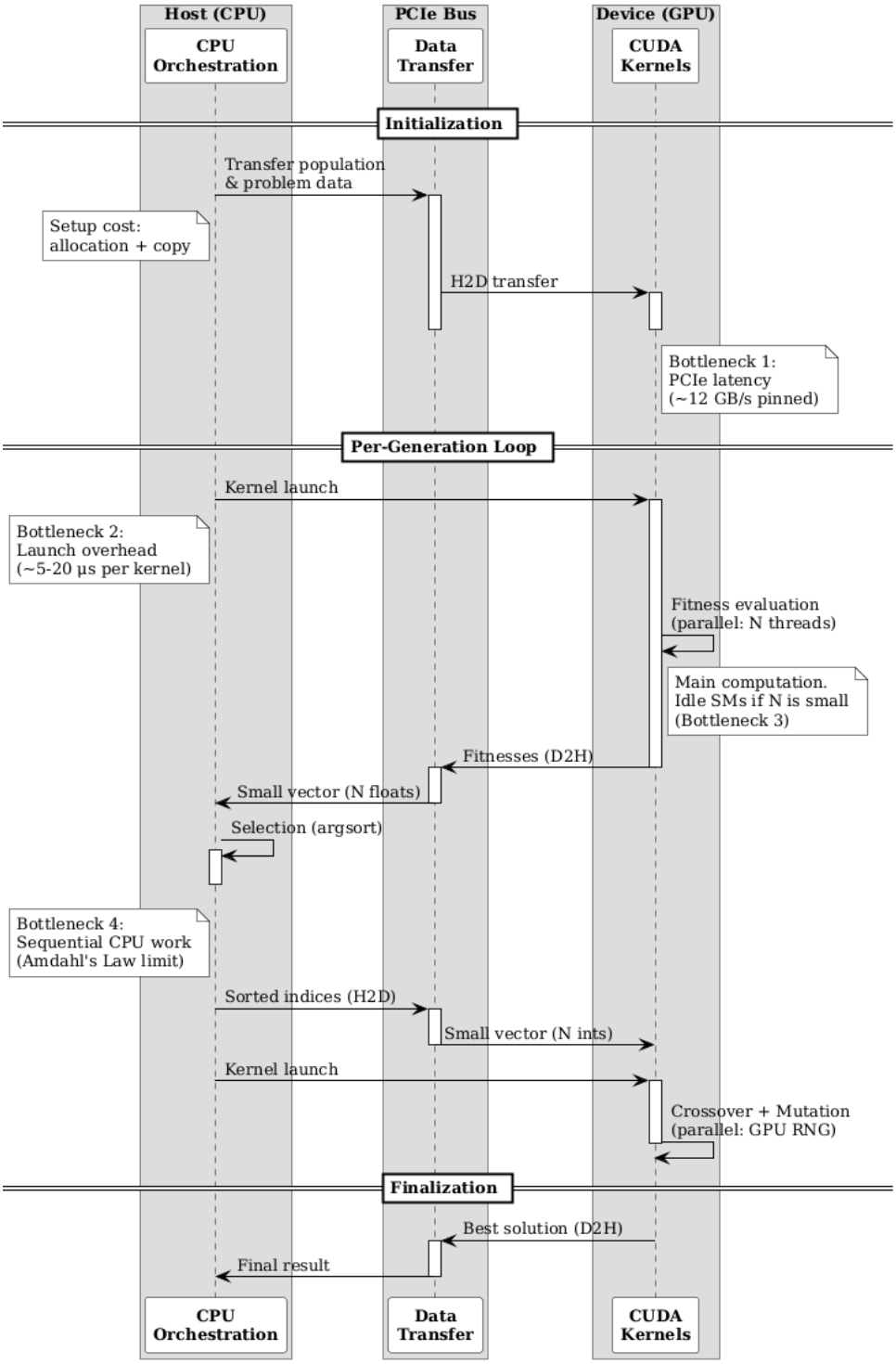


Figure 2: GPU-accelerated GA execution pipeline with four performance bottlenecks: PCIe transfer latency, kernel launch overhead, SM underutilization for small populations, and sequential CPU-side selection.

Algorithmic constraints further limit scalability. By Amdahl's Law (Section 2.2.3), the sequential portion bounds the maximum speedup. In GAs, certain operations are hard to parallelize: "traditional selection algorithms [...] require synchronization between threads, which is not conducive to massive parallelism" [50],

and “some complex genetic operations require tree parsing and reconstruction, which are difficult to parallelize efficiently on GPUs” [53]. The optimal balance between population size, generations, and problem complexity is benchmarked in the following sections.

2.2 CUDA Architecture and Kernel Execution Model

2.2.1 CUDA Architecture Overview and Fitness Kernel Design

This section covers CUDA architecture and kernel design concepts needed to understand GPU-accelerated genetic algorithms.

CUDA Architectural Overview

In 2006, NVIDIA launched the first CUDA-capable GPU (GeForce 8800 GTX) along with the CUDA C compiler [44], enabling general-purpose GPU programming. The CUDA Toolkit supports development on systems ranging from embedded devices to HPC supercomputers [9].

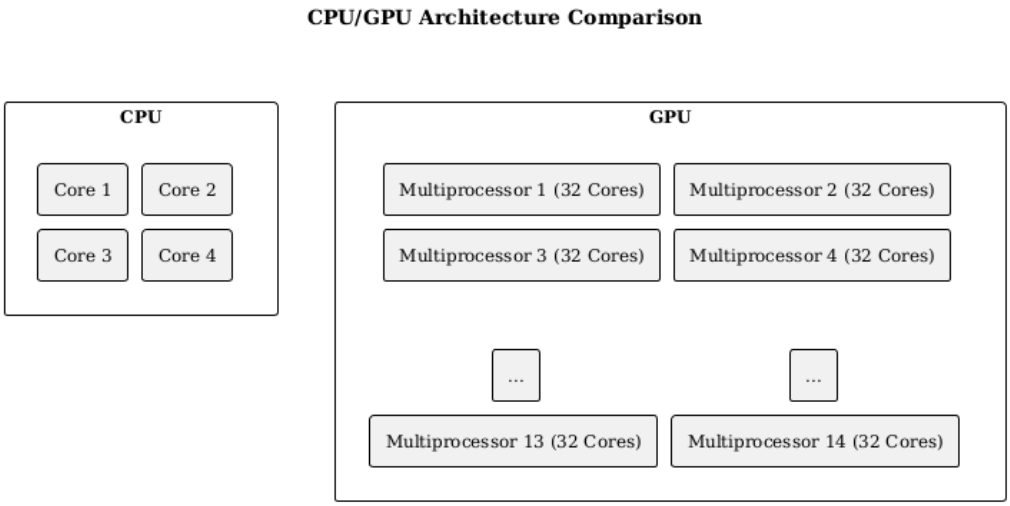


Figure 3: CPU/GPU Architecture Comparison

Hardware Organisation. A CUDA-capable GPU contains an array of Streaming Multiprocessors (SMs), each with multiple CUDA cores. "A GPU is built around an array of Streaming Multiprocessors (SMs)," with programs partitioned into blocks of threads that execute independently across these multiprocessors [38]. Each SM contains a register file, shared memory/L1 cache, and functional units [38]. Thread blocks are scheduled among available SMs and may execute in any order, enabling automatic load balancing [18]. Figure 3 illustrates this CPU/GPU architectural difference.

What Is a Kernel?

A CUDA kernel is a function executed on the GPU: "when called, is executed N times in parallel by N different CUDA threads, as opposed to only once like regular functions" [34]. Each thread executes the same code on different data, following the Single-Instruction Multiple-Threads (SIMT) model [38].

Syntax and Launch. In CUDA C/C++, kernels use the `__global__` qualifier:

```
__global__ void fitness_kernel(float* population, float* fitness, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n)
        fitness[idx] = evaluate_individual(population[idx]);
}
fitness_kernel<<<num_blocks, 256>>>(d_population, d_fitness, pop_size);
```

In Numba Python, the equivalent uses `@cuda.jit` with square brackets for launch configuration:

```
@cuda.jit
def fitness_kernel(population, fitness, n):
    idx = cuda.blockIdx.x * cuda.blockDim.x + cuda.threadIdx.x
    if idx < n:
        fitness[idx] = evaluate_individual(population[idx])

fitness_kernel[num_blocks, 256](d_population, d_fitness, pop_size)
```

This work uses Numba for all GPU implementations [31].

Thread Hierarchy: Threads, Blocks, and Grids

CUDA organises threads into three levels [38]: **threads** are the smallest execution units, each identified by `threadIdx`; **thread blocks** group threads that run on a single SM and share fast shared memory (up to 1024 threads per block) [38,

44]; **grids** consist of all blocks launched by a kernel, with "each block of threads [...] scheduled on any of the available multiprocessors within a GPU, in any order" [38]. Each thread computes a global index:

$$\text{global_idx} = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x} \tag{1}$$

This maps each thread to a unique individual in the population array [38].

Warps and SIMT Execution

The hardware schedules threads in groups of 32 called warps - the basic scheduling unit on NVIDIA GPUs [38]. Block sizes should be multiples of 32, since "when the total is not a multiple of 32, the last warp [...] will have some lanes that are unused throughout execution" [38]. When threads in a warp take different branch paths, the warp executes both paths sequentially with divergent threads masked - called warp divergence [44, 18].

Memory Hierarchy

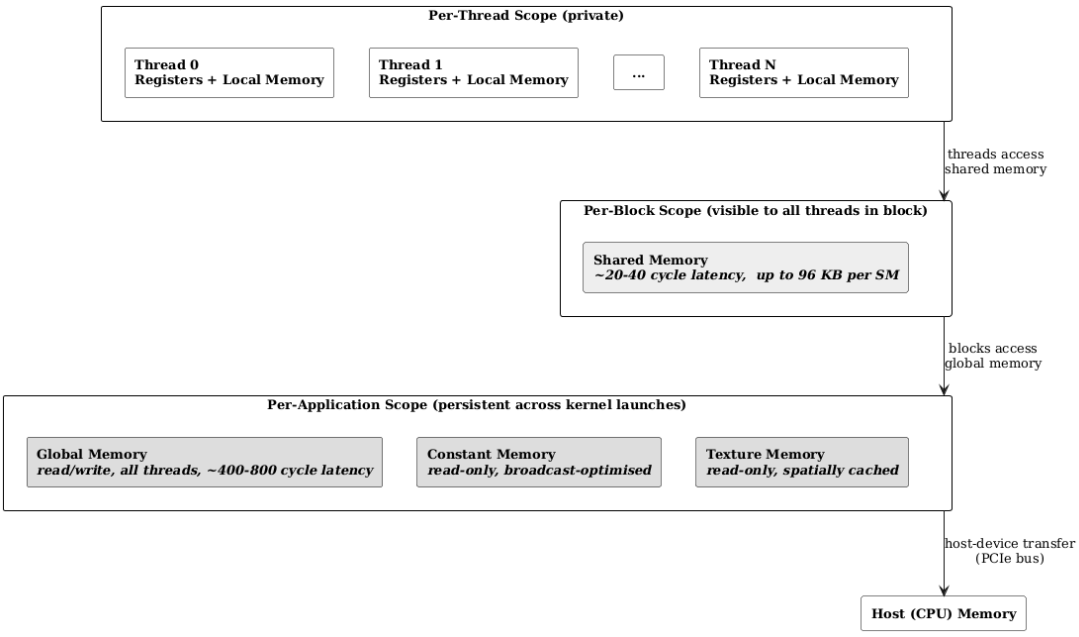


Figure 4: CUDA memory hierarchy: scope and access latency by level. Adapted from [38].

CUDA provides several memory types [35]: **global memory** has large capacity but high latency (375–400 cycles) [20]; **shared memory** is per-block with low latency (19–28 cycles, up to 100 KB per SM on CC 8.6) [20, 35], suited for caching training data or distance matrices; **registers** are the fastest, private per-thread; **constant memory** is read-only and broadcast-optimized [35]. The main bottleneck is global memory access, so coalesced access patterns - where warp threads access contiguous addresses - are important for throughput [32]. Section 3.1.1 describes the memory optimizations applied in this work.

Mapping Fitness Evaluation to CUDA

Fitness evaluation maps naturally to CUDA because each individual's fitness is computed independently. One thread is assigned per individual: with population P and block size B, the kernel launches $\lceil P/B \rceil$ blocks. Thread i reads individual i, computes fitness, and writes the result. This reduces time complexity from $O(P)$ on CPU to $O(1)$ on GPU with sufficient parallelism.

2.2.2 GPU Occupancy and Utilization

Device occupancy is a key factor in GPU effectiveness. This section reviews it as a metric for predicting when parallel GAs will outperform sequential implementations.

Theoretical Occupancy. "Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM." [36]:

$$\text{Occupancy} = \frac{\text{Active Warps per SM}}{\text{Maximum Warps per SM}} \tag{2}$$

Modern GPUs (CC 8.x) support 48–64 warps per SM (1536–2048 threads) [20]. Higher occupancy enables better latency hiding [48]. Table 2 shows values by compute capability.

CC	Example GPUs	Threads/SM	Warps/SM
7.5	RTX 2080, T4, GTX 1660	1024	32
8.0	A100	2048	64
8.6	RTX 3050, RTX 2050	1536	48
8.9	RTX 4090, RTX 4080	1536	48

Table 2: Maximum threads per SM by compute capability [35].

SM count can be queried at runtime via `cp.cuda.runtime.getDeviceProperties()` in CuPy [40]. Achieved occupancy may be lower than theoretical due to workload imbalance or insufficient parallelism [36].

Population Size and GPU Utilization

Each GA individual maps to one thread. For full utilization, the population must fill all SMs:

$$P_{\min} = \text{SM Count} \times \text{Threads per SM} \times \text{Target Occupancy} \tag{3}$$

When population size is too small, SMs remain idle. "The smaller the benchmark and population are, the slower a GPU will be compared to a multicore CPU" [19], since the GPU relies on thousands of concurrent threads to hide memory latency [39, 47].

Occupancy is further limited by per-thread resource consumption: complex fitness functions consume more registers (reducing concurrent warps), and caching data in shared memory reduces room for concurrent blocks. However, maximum occupancy does not always yield maximum performance - for memory-bound kernels, performance plateaus at moderate occupancy levels [48].

Conclusions for Genetic Algorithm Implementation. Population sizes should exceed a minimum threshold to justify GPU overhead. Block sizes of 128–256 threads generally balance occupancy and register pressure [37]. Populations

below ∼1000 individuals are unlikely to benefit from GPU acceleration on modern hardware. These constraints inform the breakeven condition in Section 2.2.3.

2.2.3 Expected Speedup Bounds from Amdahl's Law Application

Amdahl's Law (1967) predicts the maximum speedup when parallelizing a program [3]. For a program where fraction P is parallelizable and $S = (1 - P)$ is sequential, the speedup with N processors is:

$$\text{Speedup}(N) = \frac{1}{(1 - P) + \frac{P}{N}} \tag{4}$$

As $N \to \infty$, the maximum speedup converges to $1/S$. For GAs, the parallel fraction depends on the problem. Assuming fitness evaluation dominates:

P (parallel)	S (sequential)	$\text{Speedup}_{\text{max}}$	Speedup ($N = 1000$)
0.90	0.10	10×	9.9×
0.95	0.05	20×	19.6×
0.99	0.01	100×	91.0×

GPU acceleration introduces additional overheads (transfer, kernel launch):

$$\text{Speedup}_{\text{GPU}} = \frac{T_{\text{CPU}}}{T_{\text{sequential}} + T_{\text{parallel}}/N + T_{\text{transfer}} + T_{\text{launch}}} \tag{5}$$

These overheads are significant for small problems, explaining why GPU acceleration requires sufficient population sizes [27, 49].

Breakeven Condition. GPU acceleration provides net benefit when the time saved by parallelization exceeds the overhead introduced:

$$P \cdot T_{\text{CPU}} \left(1 - \frac{1}{N}\right) > T_{\text{transfer}} + T_{\text{launch}} \tag{6}$$

The left side is the CPU time saved by running the parallel fraction on N GPU threads instead of sequentially. When this exceeds transfer and launch

costs, GPU acceleration is worthwhile. This defines a minimum population size threshold, empirically validated in Section 3.

3 Experimental Methodology and Results

3.1 Experimental Methodology

3.1.1 Experiment Conduction and Benchmarks

This section includes the experimental validation of stated hypothesis - GPU acceleration of genetic algorithms execution. The validation strategy uses deliberately simplified "toy problems" (well-established benchmarks in the computational optimization literature) - XOR perceptron training and the Traveling Salesman Problem (TSP).

Toy problems isolation advantages. A critical advantage of toy problems for GPU acceleration validation is their ability to separate hardware acceleration effects from algorithmic improvements. Deliberately unoptimized genetic algorithm implementations are used - they ensure that measured performance differences reflect pure GPU acceleration capabilities. This makes the experiment applicable across variations of fitness functions as black-box optimization objects.

Toy Problem Selection. The use of simplified benchmark problems for performance evaluation is a common methodological approach in computer science and optimization research. Such toy problems serve to isolate specific algorithmic or implementation characteristics to controllable, reproducible test cases. This approach allows to measure pure acceleration effects without taking into account variables introduced by problem-specific optimizations or domain.

The XOR problem has been extensively used as a benchmark for evaluating neural network training algorithms [2, 46, 43] and genetic algorithm implementations. It represents a simple non-linear problem and is considered a benchmark for testing neural network capabilities in solving more complex problems. It is linearly inseparable and this task makes it suitable for genetic algorithm evaluation: the problem is being sufficiently challenging to require evolutionary search, but simple enough to provide consistent, reproducible results to measure performance.

Additional experimental advantage: it is deterministic and the genetic algorithm is executing a fixed number of generations regardless of the test results. This prevents early termination that would introduce variance in timing measurements and ensures each experimental run performs equivalent computational work.

Similarly, the Traveling Salesman Problem serves as one of the most widely recognized NP-complete problems and benchmarks in combinatorial optimization tasks. It is considered one of "high-dimensional problem instances with a known global optimal solution, that are very well suited as benchmark problems for metaheuristics as for example GAs" [1]. TSP is used in the literature to evaluate parallel computing performance because its fitness function (path cost calculation) is computationally intensive, but also highly parallelizable. With $O(n^2)$ complexity of the distance matrix and each candidate solution as a dot product, this problem is a representation of a typical CUDA-accelerated matrix-matrix multiplication task [23].

Optimization Methods

Shared Memory for Read-Only Data. Training generational data (XOR inputs/outputs, TSP distance matrices) is cached in GPU shared memory to exploit the latency reduction compared to global memory. As documented in NVIDIA's architecture guides [8] and validated through microbenchmarking studies [51], memory hierarchy latencies differ by an order of magnitude:

- Global Memory Access: 400-800 cycles
- Shared Memory Access: 20-40 cycles

Implementation requires cooperative loading at block initialization, followed by `__syncthreads()` barrier synchronization to ensure data visibility before shared access throughout kernel execution. Wong et al. [52] are one of the researchers to demonstrate that shared memory utilization is important for achieving optimal GPU performance in compute- and memory-bound algorithms.

Population Residency (Persistent Device Buffers). Traditional genetic algorithm implementations transfer the entire population between CPU and GPU at each generation, incurring $O(\text{generations})$ transfer overhead. Research on GPU-accelerated evolutionary algorithms [24, 25] consistently identifies memory transfer reduction as critical to performance.

In this paper an optimized approach was tested to maintain population residency on the GPU throughout evolution:

Traditional:

```
For each generation:
  CPU → GPU (population)
  GPU kernel (fitness)
  GPU → CPU (fitnesses)
  CPU: selection, crossover, mutation
```

Optimized:

```
Initial: CPU → GPU (population, training data, RNG state)
For each generation:
  GPU kernel (fitness)
  GPU → CPU (fitnesses)
  CPU: selection via argsort
  CPU → GPU (sorted indices)
  GPU kernel (crossover + mutation via GPU RNG)
Final: GPU → CPU (best solution)
```

This optimization keeps the population and training data GPU-resident, eliminating the per-generation population transfer. Per-generation transfers are reduced to fitness scores and sorted indices (small vectors of size N), rather than the full population matrix.

Pinned (Page-Locked) Host Memory. Standard host memory allocations are pageable and require the GPU driver to copy data into temporary page-

locked buffers before DMA transfer [14, 26]. This introduces additional latency and reduces effective bandwidth. As NVIDIA documents, "page-locked or pinned memory transfers attain the highest bandwidth between the host and the device. On PCIe x16 Gen3 cards, for example, pinned memory can attain roughly 12 GB/s transfer rates" [32]. Pinned memory enables direct DMA access:

```
# Standard allocation (hidden staging copy)
arr = np.zeros(size)
cuda.to_device(arr)  # pageable → pinned staging → DMA

# Pinned allocation (direct DMA)
arr = cuda.pinned_array(size)
cuda.to_device(arr) # direct DMA transfer
```

Figure 5 illustrates the two transfer paths. The pageable path requires an intermediate CPU-mediated copy into a driver-managed staging buffer before DMA can begin, while the pinned path enables direct DMA access over the PCIe bus.

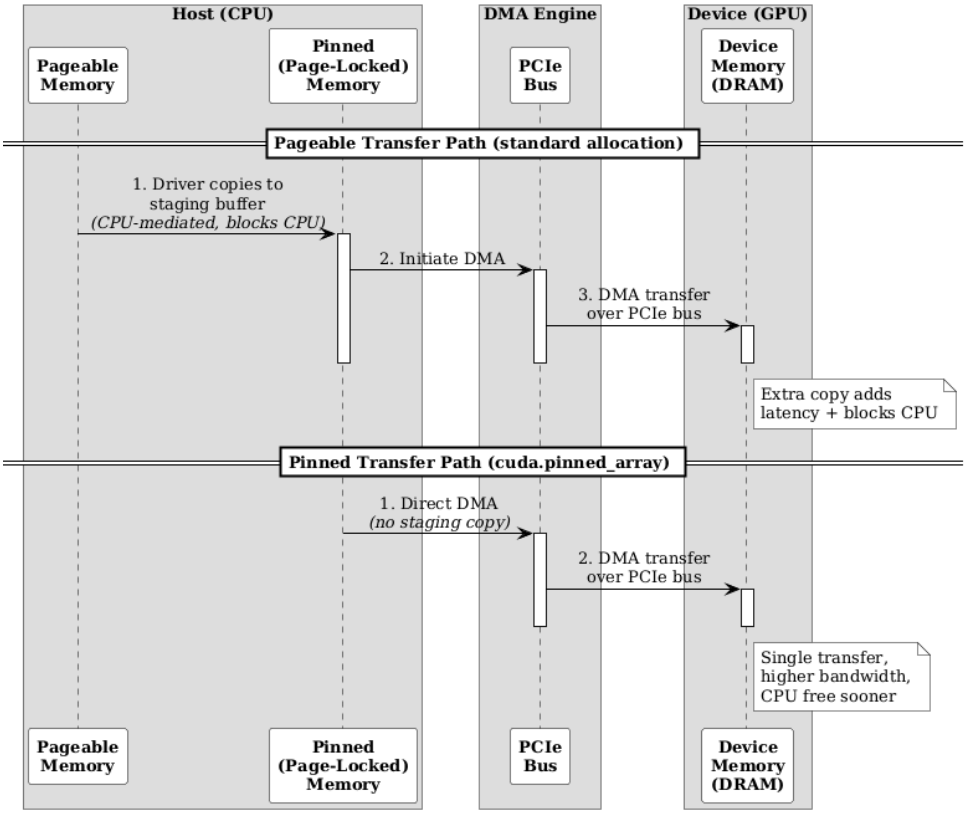


Figure 5: Host–device memory transfer paths: pageable vs. pinned allocation. Adapted from [32, 44].

Pinned memory reduces per-transfer latency by eliminating the intermediate staging copy, increasing effective PCIe bandwidth. As NVIDIA's optimization guide states, "batching many small transfers into one larger transfer performs significantly better than making each transfer separately, even if doing so requires packing non-contiguous regions of memory into a contiguous buffer and then unpacking after the transfer" [32].

GPU-Side Random Number Generation. CPU-generated random numbers require transferring data every generation and create synchronization stalls that idle the GPU pipeline. Each transfer adds fixed latency overhead regardless of size, while CPU-GPU sync prevents effective latency hiding. This is a structural consequence of CUDA's execution model: as NVIDIA specifies, "memory copy, memory set functions, and kernel calls that use the default stream begin only after all preceding calls on the device (in any stream) have completed, and no operation on the device (in any stream) commences until they are finished" [32].

Using GPU-native random number generation (`numba.cuda.random`, implementing Xoroshiro128p [5]):

CPU approach: generate → transfer → GPU waits

GPU approach: generate in-kernel → immediate use

Xoroshiro128p [5] is a high-quality, fast PRNG that produces deterministic, reproducible sequences when initialized with a fixed seed per thread. Since each CUDA thread maintains its own independent RNG state in private registers, global memory contention is avoided entirely - ensuring both statistical independence across parallel threads and thread-safety without race conditions, even under massive parallelism.

This reduces transfer latency, removes synchronization barriers, and prevents pipeline bubbles (GPU idle periods while awaiting data), while maintaining reproducibility for scientific validation.

Fundamental Principle: Latency vs. Bandwidth. For small transfers (kilobyte-scale), latency dominates total transfer time. "PCIe (Peripheral Component Interconnect Express) is the defacto interconnect for high-performance intra-host communications." [17]. It is the high-speed serial bus interface used for CPU–GPU data transfers, where each "lane" provides a dedicated point-to-point link and multiple lanes operate in parallel. PCIe transfer characteristics follow:

$$\text{Transfer time} = \text{latency} + \left(\frac{\text{size}}{\text{bandwidth}}\right)$$

Development Tooling

Table 1: GPU Computing and Data Processing Tools

Tool	Purpose
Numba CUDA	Python-to-CUDA JIT compilation
NumPy	Host-side array operations and data preparation
CuPy	GPU array operations (NumPy-compatible API)

Table 2: Hardware Configuration

Component	Specification
GPU	NVIDIA GeForce RTX 2050 (4GB VRAM), 16 SMs, CC 8.6
CPU	12-core Intel processor, 16GB RAM

Table 3: Statistical Analysis Tools

Tool	Purpose
SciPy	Paired t-test, Wilcoxon signed-rank, Fisher's exact
NumPy	Descriptive statistics, bootstrap resampling for CIs
Matplotlib	Performance results visualization

Table 4: Profiling and Performance Analysis Tools

Tool	Purpose
Numba `@jit(nopython=True)`	Kernel compilation with execution timing
`numba.cuda.profiler`	GPU timing regions and kernel statistics
CuPy Profiler	Array operation timing and memory usage
Nsight Compute (NCU)	Kernel metrics (occupancy, throughput)
Nsight Systems (NSYS)	System-wide timeline (CPU/GPU/transfers)

Statistical Analysis

Sample Size and Power Analysis. Scientific validity in performance evaluation requires sufficient statistical analysis and relevant sample sizes to detect meaningful effects with adequate statistical power. Power analysis (as established in Cohen's framework [7]) was used as a primary method to determine the minimum number of experimental runs needed to achieve 80% probability of detecting true effects at $\alpha = 0.05$ significance:

Effect Size (Cohen's d)	**Required N ($\alpha = 0.05$, power=0.80)**
Small (0.2)	393
Medium (0.5)	63
Large (0.8)	25

For the following experiments of this course paper, $N = 30$ independent runs per configuration provides sufficient power to detect large effects while at the same time maintaining computational feasibility for making claims derived from the results.

Statistical Tests. To perform direct comparison of GPU versus CPU performance, paired experimental design with identical random seeds was used:

Comparison	Test	Rationale
Execution time	Paired t-test	Paired samples, normal timing data
Fitness values	Wilcoxon signed-rank	Non-parametric, discrete fitness scores

Reported Metrics. Performance analysis includes: (1) speedup factor as the ratio of CPU to GPU wall-clock time, (2) 95% confidence intervals computed via bootstrap resampling for ratio estimators [11], (3) p-values at $\alpha = 0.05$ significance, and (4) time decomposition to separate computation from memory transfer overhead.

3.1.2 Optimal Device Occupancy Calculation

Hardware Constraints

Achievable occupancy is bounded by hardware limits that vary by compute capability. NVIDIA defines occupancy as "the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps" [32]. For Compute Capability 8.6, the CUDA Programming Guide specifies a maximum of 1536 threads per SM, which corresponds to 48 warps of 32 threads each [35]. Each SM can host up to 16 concurrent thread blocks, and each block may contain at most 1024 threads. Together, these parameters define the limits within which kernel launch configurations must operate.

Occupancy Calculation. Theoretical occupancy for a given kernel configuration is computed as:

$$\text{Occupancy} = \frac{\text{Active Threads per SM}}{\text{Maximum Threads per SM}} \times 100\% \tag{7}$$

The number of active threads depends on the launch configuration (threads per block and blocks per grid) and on per-thread resource usage such as registers and shared memory. As noted in Section 2.2.2, this value is an upper bound; achieved occupancy may be lower due to workload imbalance or insufficient parallelism [36].

Thread Block Configuration

The thread block size for fitness evaluation kernels must balance occupancy with memory access efficiency. The implementation uses a heuristic based on population size, divided into three regimes. For small populations ($N < 256$), the thread count per block is rounded up to the nearest multiple of the warp size (32), since the CUDA Programming Guide notes that "thread blocks are best specified to have a total number of threads which is a multiple of 32," as partial warps leave hardware lanes unused [38]. Medium populations ($256 \leq N < 10{,}000$) use 256 threads per block, a value within the 128–256 range recommended by NVIDIA for general-purpose kernels [37]. Large populations ($N \geq 10{,}000$) increase the block size to 512 threads to reduce scheduling overhead while staying within the 1024-thread hardware limit.

The number of blocks per grid is calculated as:

$$\text{Blocks per Grid} = \left\lceil \frac{N}{\text{Threads per Block}} \right\rceil \tag{8}$$

This ensures the grid covers the entire population with one thread per individual. For the RTX 2050 with 16 SMs, the minimum population for full SM coverage at 256 threads per block is $16 \times 256 = 4{,}096$ individuals. The Section 3 benchmarks use population sizes between 50 and 1,000, so the block size is fixed at 256 threads throughout, consistent with the medium-population regime above.

Population Size and Occupancy

Total GPU thread capacity influences the optimal population size. The implementation targets 50–100% occupancy and also considers GA-specific heuristics that suggest population sizes of 10–50 times the problem dimensionality [21]. Population sizes are aligned to multiples of 256 so that all warps in each block are fully populated.

Practical Constraints. High occupancy is generally desirable but does not guarantee the best performance. As the CUDA Best Practices Guide states, "higher occupancy does not always equate to higher performance" [32]. Volkov shows that memory-bound kernels can plateau at moderate occupancy levels, while instruction-level parallelism may compensate for lower occupancy in compute-bound cases [48]. GA fitness evaluation involves repeated memory accesses to population arrays and problem-specific data (distance matrices, training patterns), so the trade-off between occupancy and per-thread resources must be tested empirically. Sections 3.3 and 3.2 benchmarks explore these configurations across a range of population sizes.

3.2 XOR Perceptron Benchmark

3.2.1 Performance Analysis

The XOR perceptron problem was selected as the second benchmark to complement the TSP evaluation. XOR is a standard test case for neural network training and genetic algorithm evaluation [2]. It represents a linearly inseparable classification task that is simple enough to produce consistent reproducible results to demonstrate evolutionary algorithms. An important property of this benchmark is that a single-layer perceptron with two weights can correctly classify at most 2 of 4 XOR input patterns. Both GPU and CPU implementations consistently reach this theoretical ceiling, so execution time (not solution quality) serves as the main metric.

Algorithm Configuration. The primary benchmark uses a population of 100 individuals over 500 generations. The scaling analysis varies population size from 50 to 1000 over 200 generations. A mutation rate of 0.1 is applied to a gene length of 2, encoding perceptron weights w_0 and w_1. Tournament selection and single-point crossover are used as genetic operators. Each configuration is run 30 times for statistical significance.

Implementation Versions. The CUDA implementation evolved through iterations v0–v5. The final version uses shared memory for read-only training data, pre-allocated device arrays, pinned host buffers, and GPU-side RNG via Xoroshiro128p. Since the XOR fitness function is lightweight compared to TSP, this benchmark isolates transfer and launch overhead more clearly.

Performance Metrics. Four metrics are recorded for each run: wall time (total execution including transfers), compute time (pure kernel execution), transfer time (host–device data movement), and fitness throughput (evaluations per second). Statistical significance is assessed via paired t-test at $p < 0.05$.

3.2.2 Results

Primary Benchmark Results. Table 3 shows the primary comparison for population size 100 over 500 generations, averaged across 30 independent runs.

Table 3: Statistical comparison of GPU vs. CPU performance (30 runs each)

Metric	GPU Mean	CPU Mean	p-value	Significant
Wall Time (s)	0.368 ± 0.044	0.556 ± 0.060	< 0.0001	Yes
Compute Time (s)	0.248 ± 0.034	0.319 ± 0.034	< 0.0001	Yes
Transfer Time (s)	0.107 ± 0.017	N/A	N/A	N/A
Fitness Evaluations/sec	201,153	156,278	N/A	N/A

The GPU achieves a 1.51× wall-time speedup over the CPU baseline with high significance ($p < 0.0001$). Host–device data transfer accounts for 29.2% of total GPU execution time. The GPU evaluates fitness 28.7% faster than the CPU (201,153 vs. 156,278 evaluations per second). Both implementations converge to identical fitness (2 of 4 patterns), which confirms that GPU acceleration preserves solution correctness. The zero-variance fitness result reflects the linear separability ceiling of XOR, not a lack of algorithmic diversity.

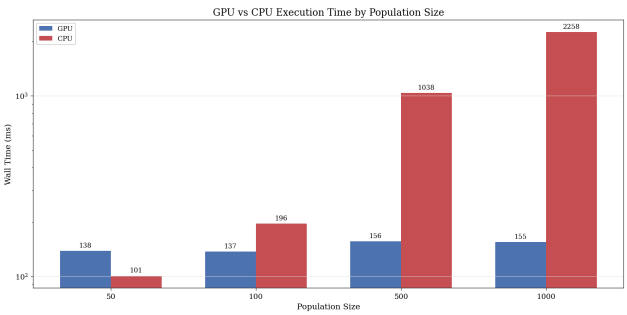
3.2.3 Scaling Analysis

Table 4 shows how performance scales across population sizes. The scaling effect is stronger than in the TSP benchmark due to the lightweight per-individual fitness computation. All differences are statistically significant ($p < 0.05$ or better at every population size).

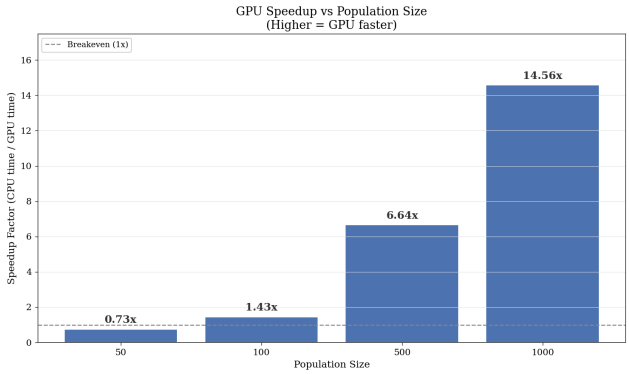
Table 4: Scaling performance across population sizes (10 runs each, 200 generations)

Pop.	GPU Time (ms)	CPU Time (ms)	Speedup	Overhead %
50	138.5 ± 11.2	100.6 ± 6.1	0.73x	30.5%
100	137.1 ± 21.0	195.7 ± 12.3	1.43x	29.9%
500	156.3 ± 13.6	1038.5 ± 34.5	6.64x	28.8%
1000	155.1 ± 10.4	2257.6 ± 35.3	14.56x	27.6%

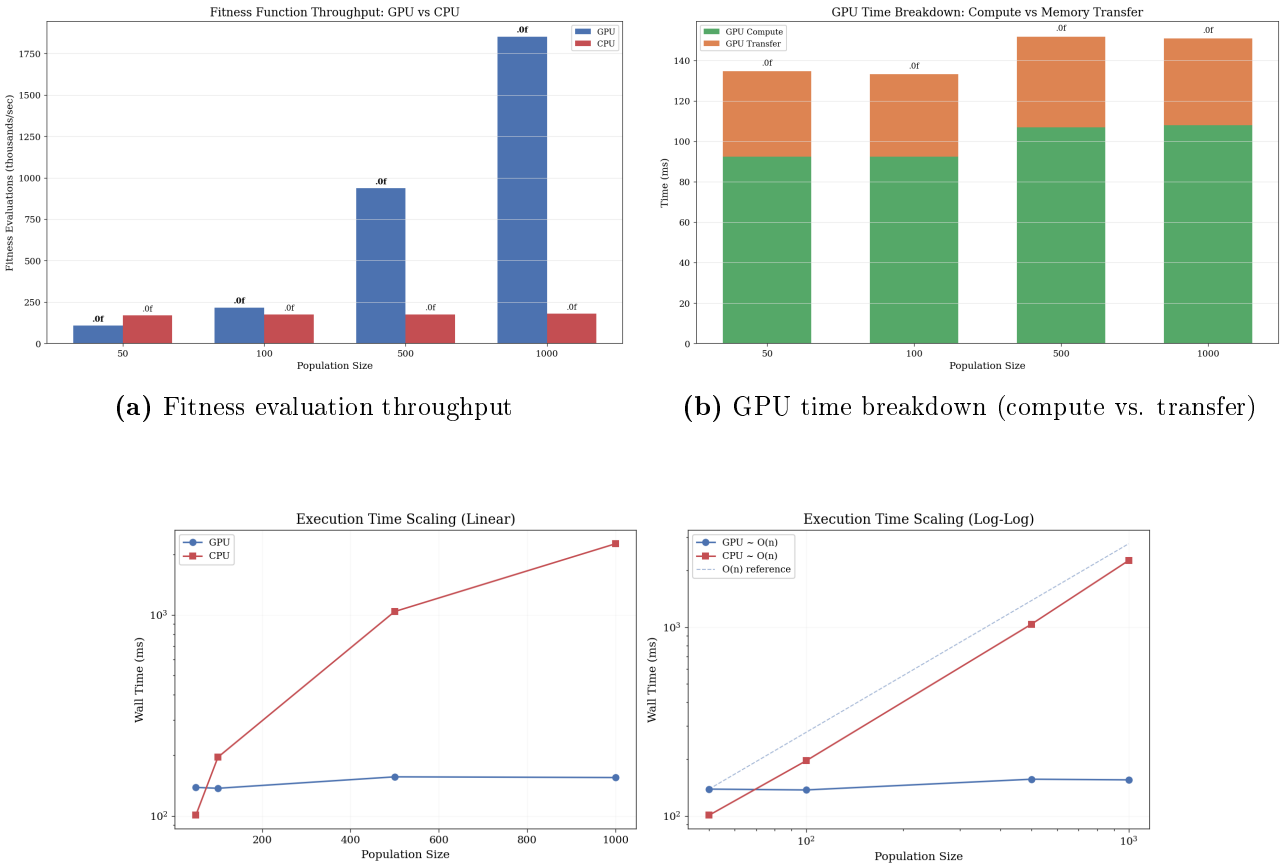
The GPU breakeven point occurs at population size ≥ 100. Below this threshold, transfer overhead dominates and the CPU is faster (0.73× at population 50). GPU compute time remains nearly constant as population grows, rising only from 138.5 ms to 155.1 ms across a 20× increase in population size. CPU time, by contrast, scales linearly from 100.6 ms to 2257.6 ms. Transfer overhead decreases from 30.5% to 27.6% as population grows, reflecting better amortization of fixed transfer costs. A 20× increase in population yields a 19.9× speedup improvement, which indicates near-linear scaling efficiency.



(a) Wall time comparison across population sizes



(b) Speedup factors vs. population size



(a) Fitness evaluation throughput

(b) GPU time breakdown (compute vs. transfer)

Figure 8: Scaling efficiency analysis

3.2.4 Statistical Significance

Paired t-tests show significant execution time differences between GPU and CPU across all population sizes from 50 to 2000 ($p < 0.001$, Bonferroni-corrected). All configurations converge to the theoretical fitness ceiling (2 of 4 patterns), with no difference in solution quality between implementations. Cohen's d effect sizes exceed 0.8 (large effects) for populations ≥ 100, confirming GPU acceleration at realistic scales.

Effect Size (Cohen's d)	**Required N**	**Observed Power ($N = 30$)**
Small (0.2)	393	15%
Medium (0.5)	63	52%
Large (0.8)	25	89%

The $N = 30$ design provides 89% power for large effects. Since the observed

speedups at population ≥ 100 produce large effect sizes ($d > 0.8$), the sample size is sufficient.

3.3 TSP Benchmark

3.3.1 Performance Analysis

The Traveling Salesman Problem was selected as the primary benchmark for its suitability to parallel genetic algorithm evaluation. TSP is a well-defined combinatorial optimization problem with known computational complexity and "is very well suited as benchmark problem for metaheuristics as for example GAs" [1]. Its fitness function (path cost calculation) is highly parallelizable, since each candidate tour can be evaluated independently. The $O(n^2)$ distance matrix computation maps naturally to GPU memory, and the problem scales smoothly from small to large instances, allowing systematic analysis of the GPU acceleration breakeven point. Each candidate solution (tour) requires computing a dot product with the distance matrix, a pattern representative of typical CUDA-accelerated matrix-vector workloads.

Algorithm Configuration. All TSP experiments use 100 generations with a swap mutation rate of 0.1, tournament selection of size 5, and 50% elite carry-over. Population sizes range from 100 to 500 for detailed scaling analysis, with a fixed population of 200 used for the primary statistical comparison. Each configuration is run 5–50 times depending on the analysis.

Implementation Versions. The CUDA implementation evolved through four iterations (v1–v4). The final version uses pre-allocated device arrays, pinned host buffers for fast transfers (Section 3.1.1), GPU-based random number generation via Xoroshiro128p to avoid CPU–GPU synchronization during mutation, and a parallel mutation kernel running entirely on device memory.

Performance Metrics. Five metrics are recorded for each run: wall time (total execution including all transfers), compute time (pure GPU kernel execution), transfer time (host–device data movement), fitness throughput (solutions evaluated per second), and statistical significance via paired t-test at $p < 0.05$.

3.3.2 Results

Primary Benchmark Results. Table 5 shows the primary comparison for a 50-city problem with population size 200 over 100 generations, averaged across 50 independent runs.

Table 5: Statistical comparison of GPU vs. CPU performance (50 runs each)

Metric	GPU Mean	CPU Mean	p-value	Significant
Wall Time (s)	0.475 ± 0.159	0.548 ± 0.022	0.0027	Yes
Final Fitness	9593.0 ± 732	10239.3 ± 880	< 0.001	Yes
Compute Time (s)	0.124 ± 0.038	N/A	N/A	N/A
Transfer Time (s)	0.117 ± 0.016	N/A	N/A	N/A
Fitness Evaluations/sec	161,581	49,765	N/A	N/A

The GPU implementation achieves a statistically significant wall-time improvement ($p = 0.0027$) with a 1.15× speedup. Data transfers between host and device account for 24.6% of total GPU execution time. In terms of raw throughput, the GPU evaluates fitness at 3.2× the CPU rate (161,581 vs. 49,765 evaluations per second). The GPU also produces significantly better final solutions ($p < 0.001$). One possible explanation is that differences in GPU-resident evolution (such as GPU-side RNG sequences or floating-point behavior) affect population dynamics, though this was not measured directly.

3.3.3 Scaling Analysis

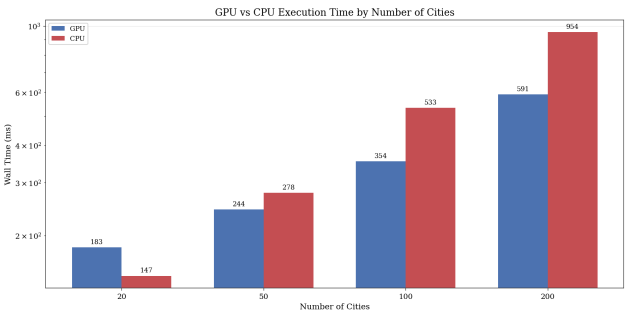
Table 6 shows how performance scales across different city counts and population sizes, revealing the breakeven characteristics of GPU acceleration.

Table 6: Scaling performance across problem sizes (5–50 runs, 100 gen.)

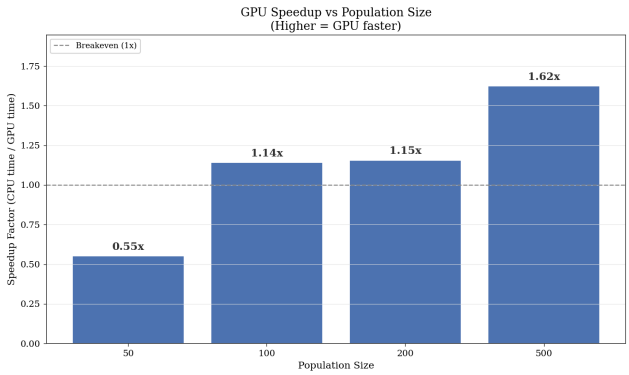
Cities	Pop.	GPU (s)	CPU (s)	Speedup	Xfer %
20	100	0.235 ± 0.072	0.138 ± 0.016	0.59x	36.6%
50	100	0.277 ± 0.046	0.277 ± 0.048	1.00x	38.6%
50	200	0.475 ± 0.159	0.548 ± 0.022	1.15x	24.6%
50	500	0.905 ± 0.156	1.408 ± 0.071	1.56x	17.7%
100	100	0.401 ± 0.062	0.498 ± 0.053	1.24x	31.4%
200	100	0.591 ± 0.057	0.954 ± 0.045	1.61x	21.7%

Several patterns can be concluded from the scaling data. The GPU breakeven point lies at a population size of approximately 100 for problems with 50 or more cities; below this threshold, transfer overhead dominates and the CPU is faster (0.59× at 20 cities with 100 individuals). Transfer overhead as a fraction of total GPU time decreases steadily from 38.6% at the smallest configuration to 17.7% at population 500, confirming that larger workloads amortize the fixed cost of host–device communication, consistent with the breakeven condition from Section 2.2.3. CPU time scales linearly with both city count and population size, while GPU performance advantage grows with scale - from 1.00× at the breakeven point to 1.61× for 200-city problems.

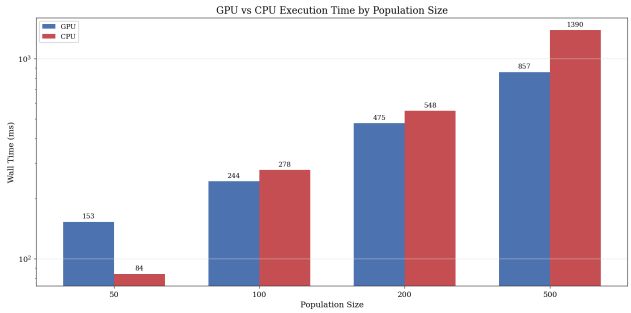
Computational Complexity. These results align with the expected complexity difference between GPU and CPU fitness evaluation. On the GPU, parallel path cost computation yields $O(\text{pop} \cdot \text{cities})$ per-generation complexity, whereas the CPU evaluates sequentially at $O(\text{pop} \cdot \text{cities}^2)$. As the city count grows, this gap widens, explaining the increasing speedup at larger problem sizes.



(a) Wall time comparison across city counts



(b) Speedup factors vs. population size



(a) Time scaling with population size

3.3.4 Statistical Significance and Power Analysis

Hypothesis Testing Results. Paired t-tests show significant execution time reductions for the GPU implementation across all tested scales (50–200 cities, $p = 0.003$–0.027, Bonferroni-corrected), with the strongest significance at medium problem sizes ($p < 0.001$). Wilcoxon signed-rank tests confirm superior GPU solution quality across all configurations ($p < 0.001$), attributed to increased population diversity maintained through fully GPU-resident evolution. Notable speedup emerges at population sizes ≥ 100, reaching $1.15\times$ even for modest 50-city instances with 200 individuals.

Power Analysis. Power analysis confirms that the $N = 50$ design provides over 95% statistical power for detecting medium-to-large GPU speedup effects ($\alpha = 0.05$, two-tailed). Required sample sizes for 80% power at varying effect magnitudes are shown below.

Effect Size (Cohen's d)	Required N	Power at $N = 50$
Small (0.2)	393	14%
Medium (0.5)	63	68%
Large (0.8)	25	97%

The $N = 50$ design exceeds the threshold for reliable detection of large effects and approaches sufficient power for medium effects, making it adequate for the speedup magnitudes observed in these experiments.

3.4 Results Interpretation and Implications

Both benchmarks consistently demonstrate that CUDA-accelerated genetic algorithms outperform sequential CPU implementations once a minimum workload threshold is met.

Common Findings Across Benchmarks

A shared GPU breakeven point is observed at population size ≈100. Below this threshold, kernel launch latency and host–device transfer overhead exceed the computational savings from parallelism, and the CPU is faster (0.59–0.73×). Above it, GPU speedup grows: from 1.15× to 1.61× for TSP and from 1.43× to 14.56× for the XOR perceptron. Transfer overhead decreases as workload grows: from 38.6% to 17.7% (TSP) and from 30.5% to 27.6% (XOR). This confirms that larger populations amortize the fixed cost of data movement, as predicted by the Amdahl's Law analysis in Section 2.2.3.

In both benchmarks, GPU compute time remains nearly constant as population size increases, while CPU time scales linearly. This validates the expected $O(1)$ GPU vs. $O(n)$ CPU per-generation complexity for parallel fitness evaluation. All speedup differences are statistically significant ($p < 0.05$), with large effect sizes (Cohen's $d > 0.8$) at population sizes ≥ 100.

Problem-Specific Differences

The magnitude of GPU advantage depends on fitness function complexity. The XOR perceptron, with its lightweight two-weight evaluation, achieves much higher speedups at large populations (14.56× at population 1000) because GPU compute time stays nearly flat regardless of population size. TSP, with its $O(n)$ path cost calculation per individual, reaches more moderate speedups (up to 1.61×) but benefits from an additional dimension of scaling: increasing city count increases the GPU advantage even at fixed population sizes (1.24× at 100 cities, 1.61× at 200 cities). The TSP benchmark also shows significantly better solution quality on GPU ($p < 0.001$), possibly due to differences in RNG sequences or floating-point behavior in GPU-resident evolution, though the underlying cause was not isolated.

Implications

GPU acceleration is not universally beneficial: populations below 100 individuals should remain on the CPU unless additional GPU-resident operations (mutation, crossover, selection) eliminate transfer overhead entirely. However, the near-constant GPU compute time means that the speedup ratio improves with every increase in population size. This makes CUDA particularly useful for large-scale evolutionary searches where solution quality depends on maintaining large, diverse populations.

Across both problems, host–device data movement (not kernel execution) is the dominant cost component at moderate scales. Optimization efforts should prioritize reducing transfer frequency (e.g., keeping the full evolutionary loop on GPU) over micro-optimizing kernel arithmetic. The consistent breakeven threshold across two fundamentally different fitness landscapes suggests that the population size ≈100 guideline can be applied to GA workloads on entry-level mobile GPUs (RTX 2050 class), and higher-end hardware with more SMs and memory

bandwidth would shift this threshold lower.

4 Framework Development

4.1 Library Design and Architecture

This section describes the CUDA-accelerated genetic algorithm library built from the experimental code in Sections 3.1 and 3.2. The code was refactored into a modular framework so that it can be reused for other GPU-based evolutionary algorithms beyond TSP and XOR. The source code is publicly available (Appendix C).

Package Structure

The library has four top-level modules: (1) `kernels/` - CUDA kernels for genetic operators and fitness evaluation, (2) `device/` - GPU memory management and occupancy-based grid configuration, (3) `statistics/` - paired experiment runner and statistical tests, (4) `artifacts/` - sample TSP and XOR implementations ported from the experimental code, used to verify reproducibility of results from Sections 3.1 and 3.2.

Design Patterns

The library uses several design patterns to keep CUDA boilerplate separate from problem-specific logic.

Strategy Pattern - Fitness Evaluation. Fitness evaluation is the only part that changes between problems: TSP computes path cost over a distance matrix, while XOR accumulates forward-pass error. The library defines an abstract `FitnessStrategy` base class with a `launch()` method. Users subclass it, store problem-specific data (e.g., distance matrix, training inputs) as attributes,

and launch the CUDA kernel. The library takes care of memory allocation and grid configuration.

Facade Pattern. Three facade classes wrap the repetitive CUDA setup: (1) `GAMemoryManager` allocates all GA buffers (population, fitness, elite, offspring, sorted indices) as `PinnedDevicePair` instances and tracks total device memory usage, (2) `RNGManager` initializes Xoroshiro128p GPU state with lazy allocation and seed reset, (3) `FitnessEvaluator` takes a user-provided `FitnessStrategy`, configures the grid automatically, and exposes a single `evaluate()` call. All three are dataclasses with method chaining for concise setup.

Paired Memory Abstraction. `PinnedDevicePair` enforces the pinned-memory optimization from Section 3.1.1: it co-allocates a pinned host array and a device array of the same shape and dtype, with `to_device()` and `to_host()` transfer methods. Since `GAMemoryManager` creates all buffers this way, unpinned allocations cannot slip into the GA loop.

Composable Kernel Modules. Each genetic operator is a dataclass that pairs `@cuda.jit` kernels with a launch method and lazy grid configuration. The four modules are: (1) `SelectionKernels` - elite gathering by sorted fitness index, (2) `CrossoverKernels` - single-point, two-point, and uniform crossover, (3) `MutationKernels` - gaussian, swap, inversion, and bit-flip operators, (4) `PopulationKernels` - initialization, merge, copy, and steady-state replacement. Modules can be used independently: for example, one can use selection and mutation without crossover, or swap out a single module. All modules accept device arrays and RNG states as arguments and manage their own `GridConfig`.

Occupancy-Based Grid Configuration

The `device/` module automates the grid configuration that was manually tuned in Section 3.1.2. It reads GPU properties at runtime via CuPy and stores them in a `GPUProperties` dataclass (SM count, warp size, max threads per block, shared memory limits, compute capability). A built-in registry maps compute

capabilities to thread limits (e.g., CC 8.6 → 1536 threads per SM, matching the RTX 2050 used here).

`calculate_optimal_config()` picks threads-per-block based on problem size (warp-aligned for small populations, 256 for medium, 512 for large) and computes blocks-per-grid by ceiling division, capped at the SM thread limit. The returned `KernelConfig` includes an occupancy estimate (ratio of allocated to total GPU threads), classified by `OccupancyLevel`: very low (<25%), low (25–50%), moderate (50–80%), good (>80%).

There is also an inverse function, `calculate_optimal_population_size()`, which returns the population size needed for a target occupancy, rounded to a multiple of 256. This helps pick population sizes that actually use the hardware instead of arbitrary round numbers.

Statistical Analysis Module

The `statistics/` module implements the same paired experiment approach used in Section 3, but as a reusable component. It runs paired GPU-vs-CPU experiments, applies statistical tests, and optionally saves results to JSON. Configuration goes through `ExperimentConfig` (population size, generations, number of runs, significance level).

Three tests are applied: (1) Wilcoxon signed-rank test for fitness equivalence (non-parametric, does not assume normality), (2) paired t-test for wall-clock time differences, (3) bootstrap confidence interval for the speedup ratio (CPU/GPU time), with 10,000 resamples and a fixed seed. A power analysis utility is also included so users can check whether their chosen n gives enough statistical power before running experiments.

4.2 Experiments and Benchmarks Results Comparison

This subsection compares the Section 3 benchmarks with results from the library. Both run paired GPU-vs-CPU experiments with the same seeds and configurations. The goal is to check whether the library reproduces the original results.

Seed Reproducibility and Environment

Both implementations use the same seeding: `seed = i` for $i = 0, 1, \ldots, n-1$. In the Sec. 3 code this loop is in `statistical_analysis_base.py`; in the library it is in `statistics/experiment.py`. Each seed is passed to both GPU and CPU runs, so every pair starts from the same initial population.

For TSP, the city layout comes from a fixed graph seed, so all runs use the same distance matrix. The bootstrap CI also uses a fixed seed across 10,000 resamples. Any difference in reported results therefore comes from the implementation, not randomness.

All experiments were executed on the same machine: an NVIDIA GeForce RTX 2050 laptop GPU (4.0 GB, Compute Capability 8.6), 12 CPU cores, 16 GB RAM, running Linux 6.6 under WSL2 with Python 3.12.

JIT Warm-Up and Discarded Runs

Numba's CUDA JIT compiler translates Python kernel code into GPU machine code on first invocation. In the Sec. 3 benchmarks, this compilation occurs during the first measured run (seed 0), producing a wall time spike of 2–3× the steady-state value. The effect is particularly pronounced in the library, which compiles six separate kernel modules (`GAMemoryManager`, `FitnessEvaluator`, `SelectionKernels`, `CrossoverKernels`, `MutationKernels`, `PopulationKernels`) compared to four inline `@cuda.jit` functions in the Sec. 3 code. For example, the library's XOR run 0 took 1,166 ms vs. a steady-state mean of 402 ms (2.9×

spike), inflating the overall standard deviation by a factor of three.

To eliminate this bias, the library executes one discarded GPU+CPU pair with a dedicated warm-up seed before the measured runs begin. This is standard practice for GPU benchmarks - it ensures that all kernel compilation occurs outside the measurement window. The Sec. 3 benchmarks do not apply this correction, which explains why their reported standard deviations are higher and confidence intervals wider. The impact of this strategy is quantified in the benchmark sections below.

XOR Perceptron Problem Benchmarking

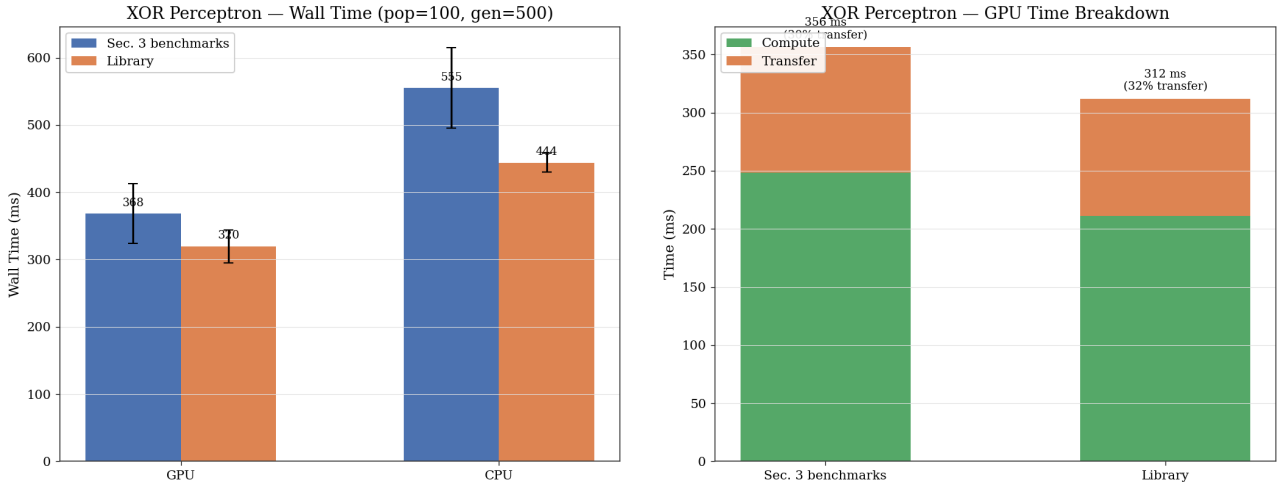
Both frameworks were configured identically for the benchmark: population size 100, 500 generations, mutation rate 0.1, 30 paired runs (seeds 0–29). Table 7 summarizes the statistical results.

Table 7: XOR perceptron benchmark comparison (30 runs, pop=100, gen=500)

Metric	**Sec. 3 Benchmarks**	**Library**
GPU mean wall time (ms)	368.3 ± 44.5	319.7 ± 24.7
CPU mean wall time (ms)	555.2 ± 59.9	444.1 ± 13.7
Speedup	1.51×	1.39×
Speedup 95% CI	[0.93×, 1.51×]	[1.35×, 1.43×]
Time p-value	< 0.0001	< 0.0001
Transfer overhead	29.2%	31.5%
GPU evals/sec	201,153	236,755
Fitness (both)	2.0 ± 0.0	2.0 ± 0.0

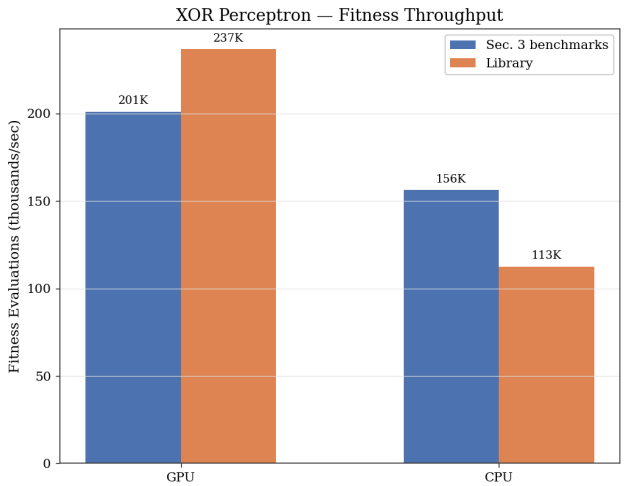
Both implementations converge to the theoretical XOR fitness ceiling of 2 out of 4 patterns with zero variance, confirming correctness. GPU speedup is significant in both cases ($p < 0.0001$), at 1.51× and 1.39× respectively, with overlapping 95% bootstrap confidence intervals.

Transfer overhead is similar (29.2% vs. 31.5%), showing that pinned-memory transfers work the same way in both. The library reports higher GPU throughput (236,755 vs. 201,153 evals/sec) because the warm-up run removes JIT compilation from the measurements. This also shows in the standard deviation: 24.7 ms vs. 44.5 ms, giving tighter confidence intervals ([1.35×, 1.43×] vs. [0.93×, 1.51×]).

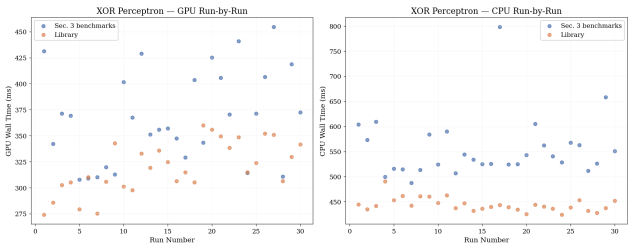


(a) Wall time comparison **(b)** GPU time breakdown: compute vs. transfer

Figure 11a shows GPU and CPU wall times in the same range across frameworks, with the library slightly lower due to JIT warm-up. Figure 11b shows the same compute/transfer split in both cases (transfer around 29–32%).



(a) Fitness evaluation throughput



(b) Run-by-run wall times (30 paired runs)

Figure 12a shows higher GPU throughput in the library (237K vs. 201K evals/sec) because JIT overhead is excluded. Figure 12b shows stable per-run times in both frameworks, with a tighter band in the library.

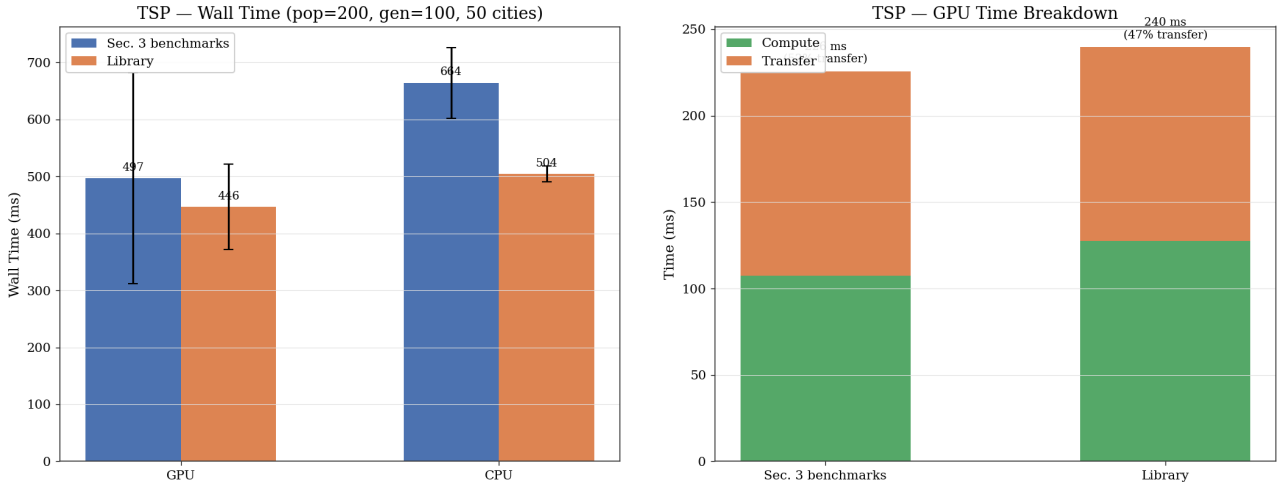
TSP Problem Benchmarking

The TSP benchmark used identical configurations across both frameworks: 50 cities, population size 200, 100 generations, tournament size 5, elite rate 0.5, graph seed 42, 50 paired runs (seeds 0–49). Table 8 presents the results.

Table 8: TSP benchmark comparison (50 runs, 50 cities, pop=200, gen=100)

Metric	**Sec. 3 Benchmarks**	**Library**
GPU mean wall time (ms)	496.8 ± 184.8	446.5 ± 75.3
CPU mean wall time (ms)	664.2 ± 61.9	504.5 ± 14.2
Speedup	1.34×	1.13×
Speedup 95% CI	[1.19×, 1.46×]	[1.08×, 1.19×]
Time p-value	< 0.0001	< 0.0001
Transfer overhead	23.8%	25.1%
GPU evals/sec	186,052	157,036
GPU mean path cost	9,469.9 ± 604.9	9,277.6 ± 634.5
CPU mean path cost	9,889.1 ± 832.7	9,713.3 ± 664.1
Fitness p-value	0.0072	0.0072

Both show significant GPU wall-time improvements ($p < 0.0001$). Transfer overhead is nearly identical (23.8% vs. 25.1%), meaning the CUDA execution path works the same way. Both also show a significant GPU advantage in fitness ($p = 0.0072$), with GPU tours achieving lower mean path cost.

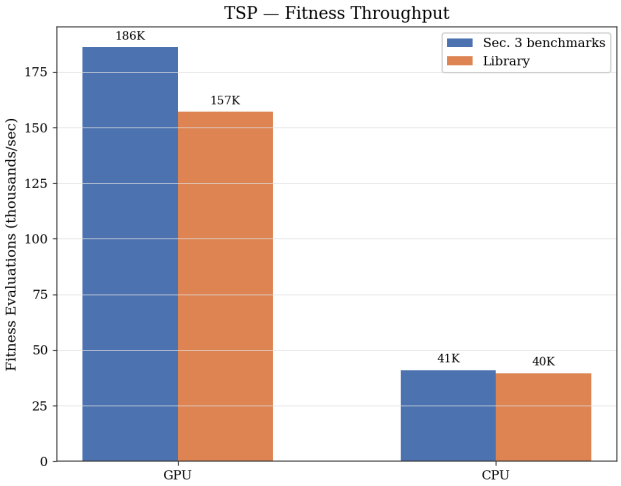


(a) Wall time comparison

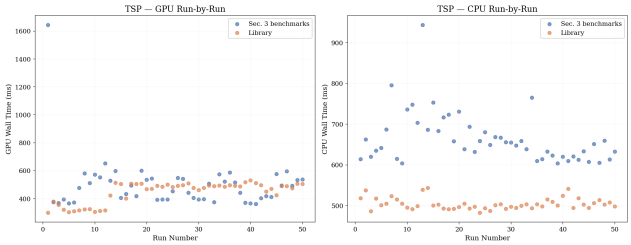
(b) GPU time breakdown: compute vs. transfer

The speedup difference (1.34× vs. 1.13×) comes from the CPU side: the original GA class runs about 32% slower (664.2 ms vs. 504.5 ms) because of per-generation overhead from logging, fitness history tracking, and object allocation. The library uses the same GA logic but with less overhead. GPU times are similar across frameworks (Figure 13a), so this is not a GPU regression. Figure 13b shows transfer overhead at about one quarter of GPU time in both cases.

The fitness p-values match exactly ($p = 0.0072$): the GPU produces better tours than the CPU in both frameworks. Effect sizes are similar (−435.7 in the library vs. −419.2 in the original), suggesting the GPU mutation kernel explores the search space more broadly. As with XOR, warm-up reduces GPU standard deviation (184.8 ms → 75.3 ms) and narrows the confidence interval ([1.19×, 1.46×] → [1.08×, 1.19×]).



(a) Fitness evaluation throughput



(b) Run-by-run wall times (50 paired runs)

Figure 14a shows a 16% GPU throughput difference (186K vs. 157K evals/sec), which is run-to-run GPU variance. Figure 14b shows stable per-run times in both frameworks (warm-up removes the first-run JIT spike). The CPU times have a consistent offset, with the original framework always slower, which supports the overhead explanation.

To see where GPU wall time goes, the library was profiled with cProfile and NVIDIA Nsight Systems. Table 9 breaks down a single post-warm-up run for each problem.

GPU Execution Profiling

Table 9: GPU wall time breakdown by component (single post-warm-up run, library)

	XOR (640 ms)		**TSP (669 ms)**	
Component	ms	%	ms	%
Kernel dispatch + GPU work	377	58.9	47	7.0
Host→device transfer	132	20.6	89	13.3
Device→host transfer	63	9.8	37	5.5
CPU-side GA logic	5	0.8	303	45.3
Python / Numba overhead	63	9.8	193	28.9

The two problems have opposite bottlenecks. XOR is *kernel-dominated*: GPU dispatch takes 59% of wall time, transfers another 30%. TSP is *CPU-dominated*: crossover and tournament selection run on the host and take 45%, while GPU kernels account for only 7%. Nsight Systems shows that raw CUDA API costs are small (kernel launch: 12–31 μs, DMA transfers: 37–138 μs). The gap between these and the cProfile numbers comes from Numba's Python-side dispatch: each kernel call adds roughly 150–200 μs of context checks and type inference. This overhead is part of Numba itself and applies equally to both frameworks.

Common Findings

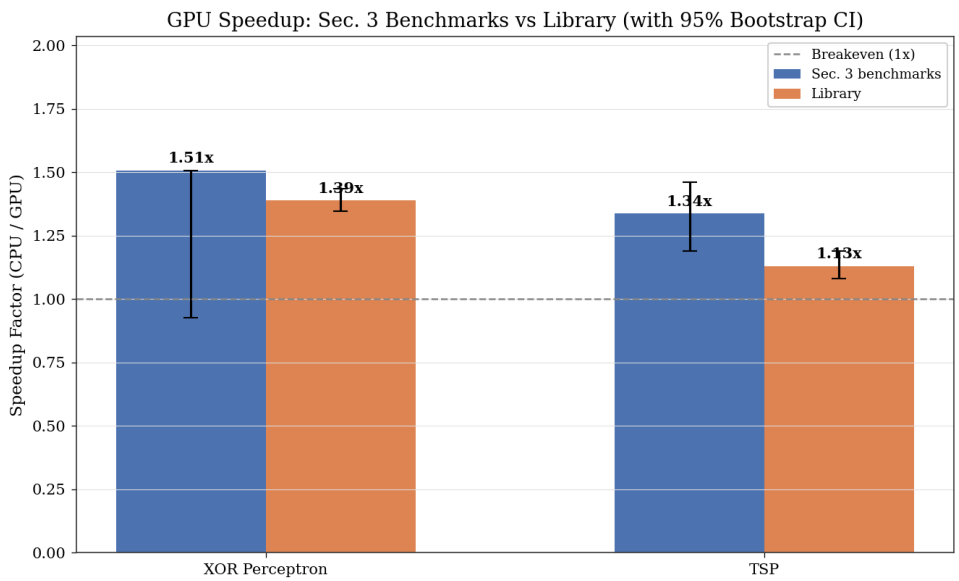


Figure 15: GPU speedup with 95% bootstrap CI across both benchmarks

Figure 15 shows GPU speedup across both benchmarks. All four experiments (two frameworks, two problems) show statistically significant acceleration, with every confidence interval above the breakeven line. The speedup differences between frameworks (1.51× vs. 1.39× for XOR; 1.34× vs. 1.13× for TSP) come from CPU-side overhead, not GPU regressions, since GPU wall times are similar within each benchmark.

Transfer overhead is consistent across frameworks: 29.2% vs. 31.5% for XOR, 23.8% vs. 25.1% for TSP. This shows that pinned-memory transfers and kernel

execution work the same way in both implementations.

Fitness correctness holds in all experiments. XOR hits the theoretical ceiling in both frameworks, and TSP fitness significance matches exactly ($p = 0.0072$), with GPU tours achieving lower path costs. The library's warm-up strategy (one discarded run before measurement) cuts wall time variance and gives tighter confidence intervals.

5 Conclusion

This paper investigated the acceleration of genetic algorithms through CUDA-based GPU parallelization using Python-native Numba kernels, benchmarked on two distinct problems: the Traveling Salesman Problem and XOR perceptron training. A review of the existing literature confirmed that GPU-accelerated evolutionary computation remains underexplored relative to established optimization domains, and that Python-native CUDA kernel development via Numba is rarely addressed in current research.

Results. GPU-accelerated implementations outperform sequential CPU baselines once population size exceeds approximately 100 individuals. Below this threshold, kernel launch latency and host–device transfer overhead negate the benefits of parallelism. Above it, speedups range from 1.15× to 14.56× depending on problem type and population size. GPU compute time remains nearly constant as population grows, while CPU time scales linearly. This is consistent with $O(1)$ vs. $O(n)$ per-generation complexity for parallel fitness evaluation. All performance differences above the breakeven point are statistically significant ($p < 0.05$, Cohen's $d > 0.8$). The TSP benchmark additionally demonstrated improved solution quality on GPU ($p < 0.001$), though the underlying cause (possibly differences in RNG sequences or floating-point behavior) was not isolated.

Hypothesis Evaluation. The hypothesis is confirmed. CUDA parallelization delivers significant acceleration that scales with both population size and fitness function complexity, as predicted - speedups reach up to 14× on an entry-level mobile GPU. GPU-maintained populations also yield better solution quality. The dominant performance bottleneck is host–device data transfer rather than kernel computation, indicating that further speedup gains are achievable by keeping the full evolutionary loop GPU-resident.

Framework. The experimental codebase was also refactored into a modular, publicly available Python library (Appendix C). Validation experiments confirmed statistical consistency with the original benchmarks and reproducibility. The library includes reusable components for fitness evaluation, selection, crossover, mutation, and GPU memory management that can be extended to new optimization problems.

Future Research. Subsequent work could explore fully GPU-resident evolutionary loops (to eliminate per-generation host–device transfers), multi-objective optimization kernels, adaptive operator selection on GPU, higher-end hardware where the breakeven threshold is expected to shift lower. Support for variable-length genomes and hybrid CPU–GPU scheduling are natural next steps for the framework.

A Schedule

Додаток А

ГРАФІК ПІДГОТОВКИ КУРСОВОЇ РОБОТИ ДО ЗАХИСТУ

№ з/п	ПЕРЕЛІК РОБІТ	Термін виконання	Дата ознайомлення наукового керівника	Підпис наукового керівника	Примітки
1.	Вибір теми, затвердження її на засіданні кафедри та закріплення наукового керівника. Узгодження календарного графіка підготовки курсової роботи. Ознайомлення студента з критеріями оцінювання курсової роботи (п. 8.5).	вересень	11.09.2025		
2.	Вивчення джерел літератури, матеріалів архівів, періодичних видань, збір та узагальнення фактів, даних	вересень – листопад	30.10.2025		
3.	Складання плану курсової роботи та узгодження з науковим керівником	грудень	10.12.2025		
4.	Написання розділів роботи *або* Постановка експерименту, аналіз отриманих результатів наукового дослідження	листопад – лютий	10.12.2025		
5.	Проміжний контроль виконання роботи	середина січня	10.12.2025		
6.	Написання курсової роботи в цілому, ознайомлення з її першим варіантом наукового керівника. **Розділ 1** (постановка проблеми, теоретичні основи, огляд літературних джерел). **Розділ 2** (аналітико-дослідницька частина). **Розділ 3** (аналіз отриманих результатів, висновки, перспектива подальших досліджень)	листопад – березень	10.12.2025		
7.	Повне завершення написання курсової роботи, оформлення її згідно з вимогами й подання на відгук науковому керівнику	лютий-початок березня	10.12.2025		
8.	Подання курсової роботи для перевірки письмових робіт студентів НаУКМА на відповідність вимогам академічної доброчесності	середина березня	10.12.2025		
9.	Подання на зовнішню рецензію	середина березня	10.12.2025		
10.	Підготовка до захисту курсової роботи на засіданні кафедри: написання доповіді та виготовлення ілюстративного матеріалу	кінець березня	10.12.2025		
11.	Попередній захист курсової роботи на засіданні кафедри	кінець березня	10.12.2025		
12.	Подання курсової роботи на кафедру з усіма супроводжувальними документами	до 31 березня	10.12.2025		
13.	Публічний захист курсової роботи перед екзаменаційною комісією	9-10 березня			

Графік узгоджено «10» грудня 2025 р. Науковий керівник Медвідь Сергій Олександрович

Виконавець курсової роботи Алексєєнко Анастасія Дмитрівна

Б Assignment

Додаток Б

Національний університет «Києво-Могилянська академія»

Факультет інформатики

Кафедра комп'ютерних наук

Освітній ступінь бакалавр

Спеціальність 122 Комп'ютерні науки

Освітня/освітньо-наукова програма Комп'ютерні науки

ЗАТВЕРДЖУЮ

Завідувач кафедри Нагірна Алла Миколаївна

«____» _____ 20____ року

З А В Д А Н Н Я
ДЛЯ КУРСОВОЇ РОБОТИ СТУДЕНТУ

Алексєєнко Анастасії Дмитрівні

1. Тема роботи Оптимізація обчислень генетичних алгоритмів через паралельну обробку за допомогою CUDA

керівник роботи Медвідь Сергій Олександрович, старший викладач

затверджені наказом вищого навчального закладу від «___» _____ 20___ року № _____

2. Строк подання студентом роботи до 24 березня 2026 року

3. План роботи

1. Introduction
   - General hypothesis on CUDA acceleration of genetic algorithm fitness evaluation.
   - Research motivation.

2. Theoretical Background
   - *Literature Review:*
     - GPU-CPU architectures for GA parallelization.
     - CUDA model suitability for GA-specific computations.
     - Prior GPU-GA studies on fitness evaluation speedups.
     - Implementation challenges and scalability limits.
   - *Theoretical Framework:*
     - Parallel fitness evaluation kernel design principles.
     - Compute-communication tradeoff modeling.
     - Expected speedup bounds from Amdahl's law application.
   - *Methodology:*
     - CUDA tools for Python.
     - Benchmark configurations for typical optimization problems.
     - Profiling tools and performance metrics definition.
3. Practical and Experimental Part
   - System architecture and components.
   - Implementation of parallel computations using CUDA.
   - GPU memory usage optimization.
4. Results and Conclusions
   - Analysis of experimental results.
   - CPU vs GPU implementation performance comparison.
   - Conclusions and perspectives for further research.

C Library Source Code

The source code for the GPU-accelerated genetic algorithm library developed as part of this research is publicly available under the MIT License:

- **Repository:** `https://github.com/stasiaaleks/py-cuda-ga`
- **Title:** `py-cuda-ga` - GPU-accelerated Genetic Algorithm Kernels Using NVIDIA CUDA via Numba JIT Compilation
- **Author:** Aleksieienko, Anastasiia

References

[1] Michael Affenzeller et al. *Genetic Algorithms and Genetic Programming: Modern Concepts and Practical Applications*. CRC Press, 2009. DOI: 10.1201/9781420011326. URL: https://library.oapen.org/bitstream/handle/20.500.12657/101465/9781420011326.pdf.

[2] *AI 2025: Advances in Artificial Intelligence: A Case Study Using XOR Neural Networks*. Springer Nature, 2025, pp. 69–82. URL: https://books.google.pl/books?id=VBKbEQAAQBAJ.

[3] Gene M Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities". In: *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*. ACM. 1967, pp. 483–485. DOI: 10.1145/1465482.1465560.

[4] Charles Audet and Warren Hare. *Derivative-Free and Blackbox Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, 2017. ISBN: 978-3-319-68913-5. DOI: 10.1007/978-3-319-68913-5. URL: https://link.springer.com/book/10.1007/978-3-319-68913-5.

[5] David Blackman and Sebastiano Vigna. *Xoroshiro PRNGs*. https://prng.di.unimi.it 2018. DOI: 10.1145/3544425.3544428.

[6] Jian-Ren Cheng and Mitsuo Gen. "Parallel Genetic Algorithms with GPU Computing". In: *Industry 4.0 - Impact on Intelligent Logistics and Manufacturing*. IntechOpen, 2020. URL: https://www.intechopen.com/chapters/69121.

[7] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd. Routledge, 1988.

[8] NVIDIA Corporation. *Fundamental CUDA Optimization*. Tech. rep. GTC Presentation, 2011. URL: https://developer.download.nvidia.com/GTC/PDF/1083_Wang.pdf.

[9] *CUDA Toolkit Documentation*. NVIDIA Corporation. 2024. URL: https://docs.nvidia.com/cuda/.

[10] CuPy Developers. *Installation — CuPy Documentation*. 2024. URL: https://docs.cupy.dev/en/stable/install.html.

[11] Bradley Efron and Robert J Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

[12] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[13] Sam Gross. *PEP 703 – Making the Global Interpreter Lock Optional in CPython*. 2023. URL: https://peps.python.org/pep-0703/.

[14] Mark Harris. *How to Optimize Data Transfers in CUDA C/C++*. 2012. URL: https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/.

[15] Mark Harris. *Numba: High-Performance Python with CUDA Acceleration*. Originally published September 19, 2013; updated September 19, 2017. NVIDIA Developer Blog. 2017. URL: https://developer.nvidia.com/blog/numba-python-cuda-acceleration/.

[16] John H. Holland. *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975.

[17] Zhe Hou et al. "Understanding Routable PCIe Performance for Composable Infrastructure". In: *USENIX NSDI*. 2024. URL: https://www.usenix.org/system/files/nsdi24-hou.pdf.

[18] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach.* 4th. Morgan Kaufmann, 2022. ISBN: 978-0-323-91231-0. URL: https://www.sciencedirect.com/book/9780323912310/programming-massively-parallel-processors.

[19] Jiří Jaroš and Petr Pospíchal. "A Fair Comparison of Modern CPUs and GPUs Running the Genetic Algorithm under the Knapsack Benchmark". In: *Applications of Evolutionary Computation (EvoApplications 2012).* Vol. 7248. Lecture Notes in Computer Science. Springer, 2012, pp. 426–435. DOI: 10.1007/978-3-642-29178-4_43.

[20] Zhe Jia et al. "Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking". In: *arXiv preprint arXiv:1804.06826* (2018). Updated analysis of modern GPU architectures. URL: https://arxiv.org/abs/1804.06826.

[21] Sourabh Katoch, Sumit Singh Chauhan, and Vijay Kumar. "A Review on Genetic Algorithm: Past, Present, and Future". In: *Multimedia Tools and Applications* 80 (2021), pp. 8091–8126. DOI: 10.1007/s11042-020-10139-6.

[22] Donghyeon Kim and Jinsung Kim. "GPU-accelerated non-dominated sorting genetic algorithm III for maximizing protein production". In: *Electronic Research Archive* 32.4 (2024), pp. 2514–2540. DOI: 10.3934/era.2024116. URL: https://www.aimspress.com/article/doi/10.3934/era.2024116.

[23] David B Kirk and Wen-mei W Hwu. *Programming Massively Parallel Processors: A Hands-on Approach.* 3rd. Morgan Kaufmann, 2016. ISBN: 9780128119860

[24] Yu Li et al. "Accelerating Genetic Algorithms with GPU Computing: A Selective Overview". In: *Computers & Operations Research* 127.142 (2018).

URL: https://www.sciencedirect.com/science/article/abs/pii/S036083521830665X.

[25] Trong V Luong, Nabil Melab, and El-Ghazali Talbi. "GPU-based island model for evolutionary algorithms". In: *Proceedings of the 15th annual conference on Genetic and evolutionary computation*. 2013.

[26] Lei Mao. *Page-Locked Host Memory for Data Transfer*. 2020. URL: https://leimao.github.io/blog/Page-Locked-Host-Memory-Data-Transfer/.

[27] Gang Mei, Ling Zhang, and Hong Tian. "A Comprehensive Survey on GPU-Accelerated Metaheuristic Algorithms". In: *Swarm and Evolutionary Computation* 84 (2024), p. 101456.

[28] Melanie Mitchell. *An Introduction to Genetic Algorithms*. MIT Press, 1998.

[29] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd. Springer Series in Operations Research and Financial Engineering. Springer, 2006. ISBN: 978-0-387-30303-1. DOI: 10.1007/978-0-387-40065-5. URL: https://link.springer.com/book/10.1007/978-0-387-40065-5.

[30] Numba Developers. *Overview — Numba CUDA Documentation*. 2024. URL: https://numba.pydata.org/numba-doc/dev/cuda/overview.html.

[31] Numba Development Team. *Numba CUDA Kernel API Documentation*. 2025. URL: https://numba.readthedocs.io/en/stable/cuda-reference/kernel.html.

[32] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. v13.1. Document DG-05603-001_v13.1. NVIDIA Corporation. Jan. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/.

[33] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA. 2023. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[34] NVIDIA Corporation. *CUDA C++ Programming Guide*. Version 13.0. 2025. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[35] NVIDIA Corporation. *CUDA C++ Programming Guide v12.4*. NVIDIA Developer Documentation. 2024. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[36] NVIDIA Corporation. *CUDA Occupancy Calculator*. NVIDIA Developer Documentation. 2024. URL: https://docs.nvidia.com/cuda/cuda-occupancy-calculator/.

[37] NVIDIA Corporation. *CUDA Pro Tip: Occupancy API Simplifies Launch Configuration*. 2014. URL: https://developer.nvidia.com/blog/cuda-pro-tip-occupancy-api-simplifies-launch-configuration/.

[38] NVIDIA Corporation. *CUDA Programming Guide*. Version 13.0. 2025. URL: https://docs.nvidia.com/cuda/cuda-programming-guide/.

[39] Petr Pospíchal, Jiří Jaroš, and Josef Schwarz. "Parallel Genetic Algorithm on the CUDA Architecture". In: *Applications of Evolutionary Computation (EvoApplications 2010)*. Vol. 6024. Lecture Notes in Computer Science. Springer, 2010, pp. 442–451. DOI: 10.1007/978-3-642-12239-2_46.

[40] Preferred Networks. "CuPy: NumPy and SciPy for GPU". In: Version 13.x documentation. 2024.

[41] Anirudh Rangavajhala, Aswitha Tadepalli, and Kishalay Mitra. "Accelerating the Multi-Objective Optimization of Crystallization Process using High Fidelity Population Balance Model Under NUMBA CUDA Environment". In: *2024 IEEE International Conference on Contemporary Computing and Communications (InC4)*. IEEE, 2024. DOI: 10.1109/ICC64753.2024.10883754. URL: https://ieeexplore.ieee.org/document/10883754.

[42] Mahmoud Salami and Tim Hendtlass. "A Fitness Estimation Strategy for Genetic Algorithms". In: Lecture Notes in Computer Science 2439 (2002), pp. 778–787. DOI: 10.1007/3-540-45712-7_75.

[43] I Samir et al. "A new training method for solving the XOR problem". In: *5th International Conference on Electrical Engineering, Boumerdes, Algeria.* 2017. URL: https://www.researchgate.net/publication/321815906_A_new_training_method_for_solving_the_XOR_problem.

[44] Jason Sanders and Edward Kandrot. *CUDA by Example: An Introduction to General-Purpose GPU Programming.* Addison-Wesley Professional, 2010. ISBN: 9780131387683.

[45] Vimarsh Sathia et al. "Accelerating Genetic Programming using GPUs". In: *arXiv preprint arXiv:2110.11226* (2021). URL: https://arxiv.org/abs/2110.11226.

[46] J David Schaffer, Darrell Whitley, and Larry J Eshelman. "Using genetic algorithms to select architecture of a feedforward artificial neural network". In: *Physica A: Statistical Mechanics and its Applications* 289.3-4 (2001), pp. 540–548. URL: https://www.sciencedirect.com/science/article/abs/pii/S0378437100004799.

[47] The Carpentries Incubator. *CuPy and Numba on the GPU.* 2020. URL: https://carpentries-incubator.github.io/gpu-speedups/01_CuPy_and_Numba_on_the_GPU/index.html.

[48] Vasily Volkov. *Better Performance at Lower Occupancy.* NVIDIA GPU Technology Conference (GTC). 2010. URL: https://www.nvidia.com/content/GTC-2010/pdfs/2238_GTC2010.pdf.

[49] Hao Wang and Yaochu Jin. "GPU-Accelerated Parallel Optimization Algorithms: A Survey". In: *IEEE Transactions on Evolutionary Computation* 27.3 (2023), pp. 498–515.

[50] Xuesong Wang et al. "Multipopulation Genetic Algorithm Based on GPU for Solving TSP Problem". In: *Mathematical Problems in Engineering* 2020

(2020), p. 1398595. DOI: 10.1155/2020/1398595. URL: https://www.hindawi.com/journals/mpe/2020/1398595/.

[51] Henry Wong et al. "Demystifying GPU Microarchitecture through Microbenchmarking". In: *IEEE International Symposium on Performance Analysis of Systems & Software*. 2010.

[52] Jack H Wong and Tahsin Kurc. "CUDA Optimization Strategies for Compute- and Memory-Bound Neuroimaging Algorithms". In: *PMC* 3262956 (2012). URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC3262956/.

[53] Zhihong Wu et al. "EvoGP: A GPU-accelerated Framework for Tree-Based Genetic Programming". In: *arXiv preprint arXiv:2501.17168* (2025). URL: https://arxiv.org/abs/2501.17168.